SYRENA (SYstème de REgulation Nouvelle Architecture)
LOT 140 Plug and Play EDR (Event Data Recorder)

EDR - 04. CAHIER DE TESTS:

REALISATION DE TEST DE L'APPLICATION EDR VERSION 1.0

par **Rafael CEREZO MARTIN**
Microsystèmes et Capteurs Autonomes (MICA)
Ecole Supérieure des Technologies Industrielles Avancées (ESTIA)

1 FONCTIONS ADDITIONNELS POUR LE DEBOGAGE 3
1.1 GESTION DE TRANSMISSION DE DONNEES SERIE POUR LE DEBOGAGE. 3
1.1.1 Fonction de Configuration du port série. 4
1.2 FONCTIONS DESTINES AU DEBOGAGE. 7
1.2.1 Fonctions ajoutes dans le Contrôleur CAN. 8
1.2.2 Fonctions ajoutes dans le PARSER 8
1.2.3 Fonctions ajoutes dans le Contrôleur de MEMOIRE 10
2 REALISATION DE TESTS UNITAIRES POUR VALIDATION DES FONCTIONS 12
2.1 AUTO VALIDATION DES FONCTIONS DU CONTROLEUR RAM. 13
2.2 AUTO VALIDATION DES FONCTIONS DU CONTROLEUR SPI. 14
2.3 AUTO VALIDATION DES FONCTIONS DU CONTROLEUR EEPROM 15
3 REALISATION DE TESTS EXTERNES A L'AIDE DE UN CAN BUS ANALYZER 16
3.1 TEST POUR VALIDER LA COMMUNICATION CAN 2.0A EXI_EDR_3 17
3.2 TEST DE LA LIBRAIRIE SPI (INTERFACE POUR LA MEMOIRE EEPROM EXTERNE) 18
3.2.1 Test des fonctions de la librairie SPI bas niveau. 18
3.2.2 Test des fonctions du contrôleur SPI. 19
3.3 TEST DU CAS D'UTILISATION «RESTAURER LA MEMOIRE» EXI_EDR_1 20
3.4 TEST DU CAS D'UTILISATION «SOLLICITER INFORMATION» EXI_EDR_7 22
3.5 TEST DU CAS D'UTILISATION «ECRIRE DES DONNEES» EXI_EDR_4, 13 ET 14 23
3.6 TEST DU CAS D'UTILISATION «LIRE DES DONNEES» EXI_EDR_5 ET 6 24
3.7 TEST DU CAS D'UTILISATION «SOLLICITER REGISTRATION» EXI_EDR_16 ET 17 25
3.8 TEST POUR VALIDER LE CHANGEMENT DE BLOCS DE MEMOIRE EEPROM 27
3.9 TEST POUR VALIDER LE TEMPS DE MISE EN «MODE OPERATIONNEL». EXI_EDR_1 29
3.10 TEST POUR MESURER LE TEMPS TOTAL DE UN CYCLE D'OPERATION DE L'EDR 30
3.11 TEST POUR VALIDER LE FONCTIONNEMENT DES COMPTEURS. 31
3.12 TEST POUR VALIDER LE TEMPS DE VIE D'UNE CASE MEMOIRE. 33
3.13 TEST AVEC UNE APPLICATION PERMETTANT DE SIMULER LA COMMUNICATION AVEC L'ECU 36
4 REALISATION DE TESTS ENVIRONNEMENTAUX. 39
4.1 TEST TEMPERATURE 39
4.2 TEST ETANCHEITE 45
4.3 TEST VIBRATIONS 45
5 APPENDICE 46
5.1 APPENDICE 1. CARTE EDR 46
5.2 APPENDICE 2. PROBLEME DE CLOCK AVEC COMPILATEUR XC8 47

Document		
Titre	EDR - 04. CAHIER DE TESTS: REALISATION DE TEST DE L'APPLICATION EDR V1.0	
Fichier	20130918_ESTIA_SYRENA_L140_EDR_04_Cahier Test_v1.0.docx	
Auteur	Guillaume TERRASSON [GT]	ESTIA
	Rafael CEREZO MARTIN [RCM]	ESTIA
Relecteurs	Renaud BRIAND [RB]	Aquitaine Electronique

Suivi des Modifications			
Edition	**Date**	**Auteur**	**Description des modifications**
1	10/04/2013	GT	Création du document
2	3/6/2013	RCM	Fonctions additionnels pour le débogage
3	7/6/2013	RCM	Réalisation de tests unitaires pour validation des fonctions
4	9/6/2013	RCM	Réalisation de tests externes à l'aide de un CAN Bus Analyzer
5	1/7/2013	RCM	Test des cas d'utilisation et validation du changement de blocs de mémoire EEPROM
6	9/7/2013	RCM	Test pour mesurer le temps total de un cycle d'opération de l'EDR
7	15/7/2013	RCM	Test pour valider le temps de vie d'une case mémoire
8	23/7/2013	RCM	Test pour valider le fonctionnement des compteurs
9	21/08/2013	RCM	Test en température (CONFIGURATION 1) et (CONFIGURATION 2)
10	2/9/2013	RCM	Test avec une application permettant de simuler la communication avec l'ECU.
11	6/9/2013	RCM	Test en température (CONFIGURATION 3)

GLOSSAIRE	
Terme	**Description**
PnP	Plug'n Play
ECU	Engine Control Unit
EDR	Event Data Recorder
E/W	Erase/Write

1 Fonctions additionnels pour le débogage

Une fois les fonctions métier de l'EDR développés on va ajouter une série de fonctions additionnelles qui vont nous permettre de réaliser de façon automatique des tests de validation. Pour cela on va s'aider d'une de les 4 sorties disponibles pour la configurer comme une **ligne de transmission série** qui nous permettra envoyer des lignes de log et observer dans un terminal le déroulement de nos tests.

1.1 Gestion de transmission de données série pour le débogage.

Voyons à continuation une série d'**observations sur le hardware** utilise.

Il se trouve que les lignes destines à gérer le port série sur le microprocesseur PIC18F62K80 ne sont pas câblés ver aucun connecteur de sortie dans la carte EDR.

- 14 - RC6/CANTX/TX1/CK1/CCP3
- 15 - RC7/CANRX/RX1/DT1/CCP4

Comme on a vu précédemment, au moment de la conception de la carte il se trouve qu'on était réservé 4 connecteurs perforées connectées à 4 pins du microprocesseur, deux sur le PORTA (RA0 et RA1) et deux sur le PORTB (RB4 et RB5). Donc, l'idée est d'utiliser une de ces sorties pour la configurer comme ligne de Transmission Série TX.

Cette ligne se sortie logique digital TTL doit être converti pour avoir une amplification du signal de façon à pouvoir être connecte vers le port série de un PC (logique 0 : +3 V to +15 V, logique 1 : -3 V to -15 V). Cette conversion est réalisé typiquement par le driver MAX232N le quel on trouvera dans la carte PICDEM CAN-LIN 3. Le schéma suivant montre comme réaliser la connexion pour obtenir une line de transmission série TX à partir de la **sortie RB4** disponible sur la carte EDR.

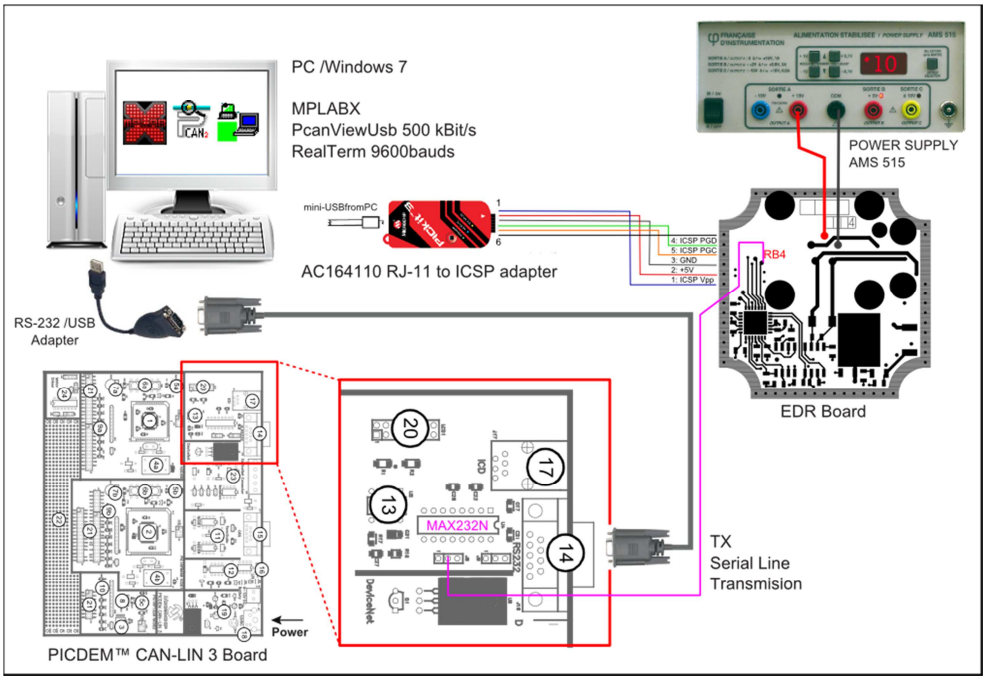


Figure: Connexion de la carte EDR avec la ligne série TX pour le débogage.

Une fois les aspects matériels on était présentés, on va citer les **observations logiciel** nécessaires pour pouvoir mettre en œuvre la transmission des données série pour le débogage.

Tout d'abord indiquer que étant donné que on n'utilise pas les pins assignes pour le bus série du microcontrôleur on ne pourra pas non plus utiliser les fonctions de la librairie UART du PIC18. Dans notre cas on utilisera l'implémentation «software» des fonctions UART **sw_uart**. Ces fonctions vont nous permettre de **configurer autres pins d'autres ports** pour les considérer comme lignes de réception RX et transmission TX.

La librairie fournie par xc8 contient des fonctions pour initialiser les ports et pour réaliser la transmission/réception des données. Etant donné que on n'utilise pas la routine hardware de initialisation la quel utilise le registre BAUDCON1 pour fixer la vitesse de communication série, le programmateur doit **ajouter des routines pour créer des retards** sur chaque valeur **de bit** qui seront en accord avec la vitesse de communication série désirée. Le retard devra être le plus précis possible à fin de que port série du PC réalisé l'échantillonnage des bits correctement.

Pour plus d'information sur ce sujet consulter le lien:

http://www.eewiki.net/display/microcontroller/PIC18+Software+UART+Timing+Requirements

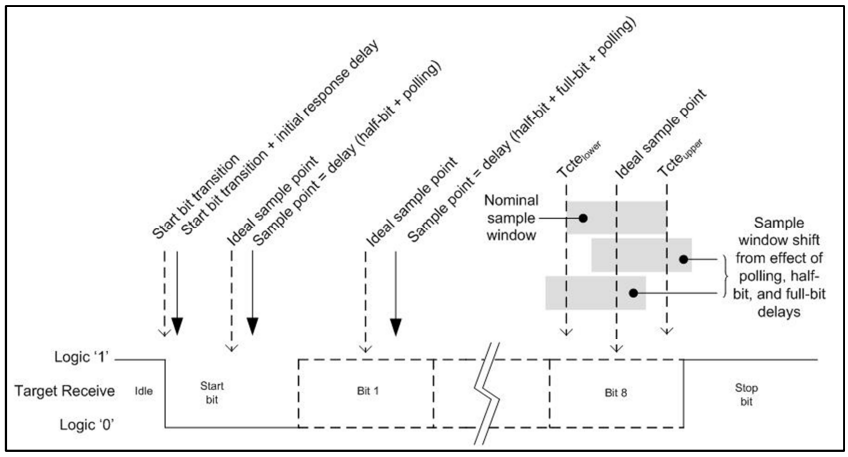


Figure: Localisation du point d'échantillonnage pour la détection d'un bit.

Il se trouve que dans la version actuelle de MPLABX les fonctions **sw_uart** qui se trouvent dans le répertoire C:\....\xc8\v1.12\sources\pic18\plib\SW_UART **présentent certaines erreurs** qu'il faudra corriger pour pouvoir mettre en route la transmission série.
On renommera ces fonctions pour éviter une confusion avec celles d'origine.

Dans notre projet, ces fonction sont décrites dans le fichier **UARTc/.h** dans le répertoire **src/ModulesPIC/.**

1.1.1 Fonction de Configuration du port série.

On commence par la fonction **OpenUART()** la quel on nommera **SW_UART_Open()** et qui sera utilisé pour établir les pins avec lesquels on va connecter nôtres lignes de communication RX et TX. Pour le débogage on aura **uniquement besoin de la ligne de transmission TX** que selon le schéma des connexions précèdent on a établi de la connecter au **pin B4.**

```
void SW_UART_Initialize(void)
{
    TRIS_SWTXD.TRISbit_SWTXD = 0x0;    // TRISBbits.TRISB4 as input for TRANSMISION
    SWTXD.SWTXDpin           = 0x1;    // PORTBbits.RB4. Rise LEVEL.

    TRIS_SWRXD.TRISbit_SWRXD = 0x1;    // TRISBbits.TRISB5 as ourput
    SWRXD.SWRXDpin           = 0x1;    // PORTBbits.RB5
}
```

1.1.1.1 Fonction de Transmission de données par le port série.

A la date actuel, la fonction pour la transmission de un byte par le port série libre dans la librairie standard de xC8 dans la version 1.12 **WriteUART()** présent des erreurs de programmation reconnus par Microchip. Ce fonction ne gère pas les envois des bits de START et STOP ce qui rend la communication série impossible.

Pour corriger cet erreur on va créer une nouvelle fonction nomme **SW_UART_Write()** permettant placer les bits nécessaires pour la transmission de un byte dans le bus série (à travers du pin configure par le programmateur RB4).

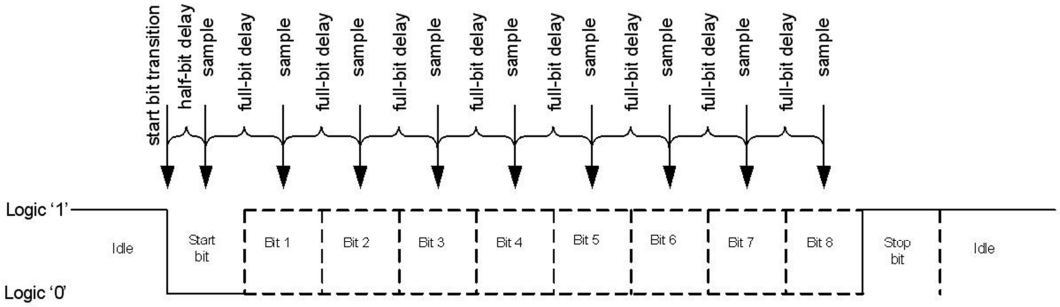


Figure: Séquence de bits pour la transmission de un byte avec retards et points d'échantillonnage

Une fois valide la fonction de transmission d'un byte on va créer un autre nomme **SW_UART_puts()** qui vas nous permettre **d'envoyer un chaine de caractères** la quel s'appuie sur la précèdent fonction pour réaliser la transmission byte a byte.

```
void SW_UART_puts(const unsigned char *data)
{
  do
  {    // Transmit a byte
    SW_UART_Write(*data);
  } while( *data++ );
}
```

1.1.1.2 Fonction de Delay pour établir les retards des bits.

Pour utiliser les fonctions **sw_uart** de transmission et réception série selon les spécifications du compilateur xC8, il est nécessaire d'ajouter 3 fonctions pour générer les retards de bits. Selon les commentaires décris dans le code ces retards doivent tenir compte de un certain nombre de cycles instruction (Tcy) pour établir la duration.

Etant donné que, par exemple, nous on utilise un **oscillateur de 10MHz** et on désire communiquer par port série a une vitesse de communication de **9600 baud** (9600 bit/seconde), on peut déduire la valeur de :

1 **bit time** = 1000 msec/sec / 9600 bit/sec = 0,104166666 msec/bit

Selon ces valeurs, on peut déjà déterminer la valeur des retards selon les formules fournies.

Fonction	Tcy for overhead	Tcy call to DelayTXBit	Tcy for return	Retard en Tcy	Valeur des formules
DelayTXBitUART	8	2	2	`(((2*OSC_FREQ)/(4*BAUDRATE))+1)/2 - 12`	`260,916667 - 12`
DelayRXHalfBitUART	5	2	2	`(((2*OSC_FREQ)/(4*BAUDRATE))+1)/2 - 9`	`260,916667 - 9`
DelayRXBitUART	10	2	2	`(((2*OSC_FREQ)/(4*BAUDRATE))+1)/2 - 14`	`260,916667 - 14`

En principe nous on va utiliser uniquement la transmission donc il nous suffit d'écrire la fonction **DelayTXBitUART()** que nous permet d'établir un retard de **104,166 µsec** selon le calcul précèdent.

Il se trouve que dans xC8 ils existent déjà définies des macros qui vont permettre d'établir des retards pour des millisecondes et pour des microsecondes, à condition de ajouter un #define avec la valeur de la fréquence de l'oscillateur utilise (dans notre cas `#define _XTAL_FREQ 10000000`). .

```
// NOTE: To use the macros below, YOU must have previously defined _XTAL_FREQ
#define __delay_us(x) _delay((unsigned long)((x)*(_XTAL_FREQ/4000000.0)))
#define __delay_ms(x) _delay((unsigned long)((x)*(_XTAL_FREQ/4000.0)))
```

Ces fonctions font appel à la fonction de plus bas niveau, **_delay()** que va calculer le nombre de cycles instruction (Tcy) nécessaires pour réaliser le retard sollicite à partir de la duration de une instruction selon la formule :

_XTAL_FREQ/4000000.0 = 10000000 / 4000000.0 = **2.5 µsec**

Valeur qu'on obtient également par la définition :

Tcy period of one instruction cycle = 1/(Fosc*4) = 1 / (10000000 * 4) = 0,000000025 msec = **2.5 µsec**

Par exemple, la fonction _delay_us(4) pour obtenir un retard de 4µsec déterminera une valeur de 4 * 2,5 = 10 Tcy

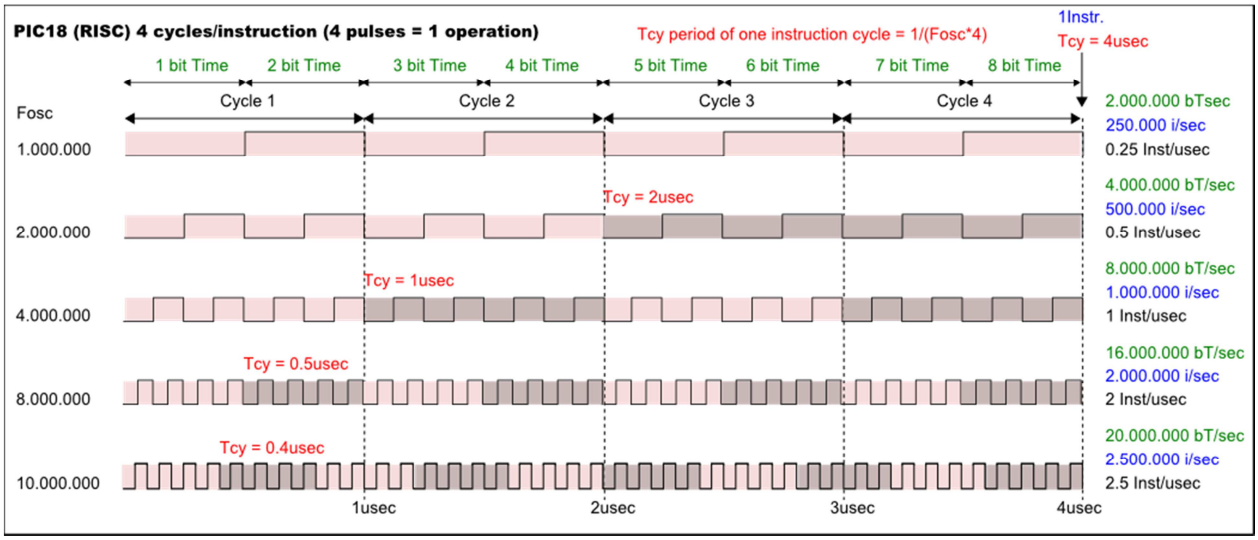


Figure: Valeur de nombre de cycles instruction (Tcy) selon la fréquence d'oscillateur.

Il se trouve que pour la valeur de `_XTAL_FREQ = 10000000` la fonction `__delay_us` ne donnent pas les bons temps mesures à l'oscilloscope, et donc les caractères envoyés ne sont correctement reçus dans le port série du PC.

On décide d'utiliser la fonction de bas niveau _delay() et trouver avec une série d'essais la valeur qui va générer un delay proche de celle désire au même temps que on observe si le byte 0x55 et bien reçu par le PC.

```
/// _delay(500); 208.0us (9600) 0x55 >> CEC6 CECE CEC6 CECE     >> ERREURS
/// _delay(550); 230.0us (9600) 0x55 >> CEC6 CECE CEC6 CECE     >> ERREURS
/// _delay(248); 109.0us (9600) 0x55 >> D5 55 D5 55             >> ERREURS (Selon formule)
/// _delay(246); 108.4us (9600) 0x55 >> 55 55 55 55 55 55 55 55
/// _delay(244); 107.6us (9600) 0x55 >> 55 55 55 55 55 55 55 55
/// _delay(240); 106.0us (9600) 0x55 >> 55 55 55 55 55 55 55 55
/// _delay(235); 104.0us (9600) 0x55 >> 55 55 55 55 55 55 55 55
/// _delay(234); 103.6us (9600) 0x55 >> 55 55 55 55 55 55 55 55
```

Donc, on observe que pour envoyer les caractères correctement et s'approcher du retard de **104,166 µsec** désire la valeur plus convenable à utiliser comme paramètre es **_delay(235);** (Valeur des formules 260,9 – 12=248,9)

1.2 Fonctions destines au débogage.

Afin de pouvoir différentier les fonctions propres au métier EDR de celles qui vont servir uniquement au débogage, on a adopté la convention de inclure dans ces dernier le mot **_DEBUGGER_** dans le nom de la fonction.

Pour pouvoir lancer ces fonctions depuis l'extérieur, qui pourront nous servir dans la phase de débogage et maintenance, on a ajouté dans le PARSER une nouvelle requête à laquelle l'EDR sera capable d'accepter des trames afin de réaliser des opérations de débogage ou consultation de l'état de l'EDR. Celles trames pourront être envoyés de façon:

- **Manuel** par un administrateur a travers d'une application qui gère un analyseur de bus CAN
- **Automatique**, par l'ECU ou autre application de maintenance.

On appellera la trame de réception de cette nouvelle requête **Admin_Query** , et après exécution des fonctions sollicités par la requête, l'EDR finira pour retourner une trame que on nommera **Admin_Reply.**

On a adopté une convention pour établir l'information qu'on transmet dans les trames de requête (**Query**) et de celle qu'on reçoit dans les trames de réponse (**Reply**). Essentiellement consiste à utiliser:

- Les **premiers 4 bytes** de la trame CAN pour indiquer le **Code de 'Opération**. Autant dans les trames de requête comme de réponse.
- Les **derniers 4 bytes** de la trame CAN
  - Dans les trames de requête seront un **paramètre** pour passer à la fonction de débogage.
  - Dans les trames de réponse seront le **résultat** retourne par la fonction de débogage.

	ID	Byte 0	Byte 1	Byte 2	Byte 3	Byte 4	Byte 5	Byte 6	Byte 7
	11	1 31				0 31			0
Admin_Query →		ID_AQ	EEPROM_CLEAR_BANK		0x0	_bank_Selected			
Admin_Reply ←		ID_AR	EEPROM_CLEAR_BANK		0x0	**CRC** EEPROM **STORED** or 0x0(CRCs not coherents)			
Admin_Query →		ID_AQ	MEMORY_RESTORE_RAM		0x1				
Admin_Reply ←		ID_AR	MEMORY_RESTORE_RAM		0x1	**CRC** EEPROM **STORED** or 0x0(CRCs not coherents)			
Admin_Query →		ID_AQ	GET_BANK_ACTIVE		0x2				
Admin_Reply ←		ID_AR	GET_BANK_ACTIVE		0x2	_bank_Active			
Admin_Query →		ID_AQ	GET_BANK_CRC		0x3	_bank_Selected			
Admin_Reply ←		ID_AR	GET_BANK_CRC		0x3	**CRC** EEPROM **STORED**			
Admin_Query →		ID_AQ	GET_COUNT_REC_QUERYS		0x4				
Admin_Reply ←		ID_AR	GET_COUNT_REC_QUERYS		0x4	countQuerys			
Admin_Query →		ID_AQ	GET_COUNT_REC_OK		0x5				
Admin_Reply ←		ID_AR	GET_COUNT_REC_OK		0x5	countFinish			
Admin_Query →		ID_AQ	GET_ERROR_STATUS		0x6				
Admin_Reply ←		ID_AR	GET_ERROR_STATUS		0x6	_error_Status			
Admin_Query →		ID_AQ	RESET_ERROR_STATUS		0x7				
Admin_Reply ←		ID_AR	RESET_ERROR_STATUS		0x7	_error_Status			

Figure : Liste des trames CAN utilises pour le débogage.

Il est donc nécessaire d'établir des nouveaux identificateurs CAN à fin de différentier ce type de requêtes :

- **ID_AD**: ID Admin Query, Identificateur pour les trames de requête.(Valeur **0x404**)
- **ID_AR**: ID Admin Reply, Identificateur pour les trames de réponse. (Valeur **0x304**)

Pour pouvoir intégrer ce mécanisme on a dû réaliser de façon progressive quelques ajustements dans notre projet que on va expliquer a continuation.

1.2.1 Fonctions ajoutes dans le Contrôleur CAN.

Initialement, pour pouvoir accepter ces trames on doit ajouter dans le **contrôleur CAN** quelques opérations nécessaires:

- Modifier la fonction de initialisation **EDR_CAN_CONTROLLER_Initialize** pour qu'elle accepte un **paramètre additionnel** avec l'identificateur crée pour les trames Admin_Query , pour **ajouter un filtre** qui va envoyer ces trames vers le buffer 2.

```
// Buffer 2 will receive also Recording_Query(IDQ5)
    CANSetFilter(CAN_FILTER_B2_F3, aFilter_IDQA, CAN_CONFIG_STD_MSG);
```

- Ajouter deux nouvelles fonctions pour le paquetage et dépaquetage de la trame;
  - EDR_CAN_DEBUGGER_Get_Command_Value: (Unpack) Récupère le Code d'opération et le paramètre à passer, et **appelle à la fonction** de débogage..
  - EDR_CAN_DEBUGGER_Send_Admin_Reply: (Pack) **Compose la trame** avec le Code d'opération et le résultat retourne par la fonction et l'**envoi** à travers du bus CAN.

1.2.2 Fonctions ajoutes dans le PARSER

A continuation on doit ajouter une autre condition dans la fonction EDR_PARSER_Manage_Querys pour traiter les trames que seront récupérés avec l'identificateur Admin_Query. Des que une trame de cet type est réceptionne on appellera en suite à la fonction EDR_PARSER_DEBUGGER_Admin_Query en le passant comme paramètre l'identifiant qui doit retourner après avoir réalisé la fonction

A l'appelle de la fonction EDR_PARSER_DEBUGGER_Admin_Query la premier action à réaliser est la récupération du code d'opération et le paramètre à passer, et pour cela on utilise la fonction crée précédemment dans le contrôleur CAN EDR_CAN_DEBUGGER_Get_Command_Value.

L'action suivante consiste à traiter la commande reçue selon le code. Pour cela, on a établi préalablement toute une liste de **codes d'opération acceptes**, et selon la commande reçu on appellera a une fonction pour réaliser une opération spécifique ou bien modifier ou récupérer un paramètre.

```
#define EEPROM_CLEAR_BANK           0xC1EABA00
#define MEMORY_RESTORE_RAM          1
#define GET_BANK_ACTIVE             2
#define GET_BANK_CRC                3
#define GET_COUNT_REC_QUERYS        4
#define GET_COUNT_REC_OK            5
#define GET_ERROR_STATUS            6
#define RESET_ERROR_STATUS          7
```

Apres avoir réalisé l'opération, on prépare la réponse de retour et on l'envoi en utilisant la fonction crée précédemment dans le contrôleur CAN EDR_CAN_DEBUGGER_Send_Admin_Reply.

Certains codes d'opération seront traites par le contrôleur de mémoire (EDR_MEMORY_CTR) et certains autres par le PARSER lui-même (EDR_PARSER_CTR).

On continue dans ce point pour expliquer les fonctions de débogage ajoutes dans le PARSER que principalement vont servir à manipuler une variable interne utilise pour le **traitement des erreurs** qui peuvent apparaitre dans les différents modules: communication CAN, communication SPI, erreurs mémoire, erreur du métier EDR, etc.

La variable _error_status

Cette variable de **32 bits** va nous permettre de registrer le long du fonctionnement de l'application les possibles erreurs qui vont apparaitre. La table suivante montre une liste de premières possibles erreurs:

Bit	Taille	
00	ERROR_EMC_NO_ZONE_ACTIVE	
01	ERROR_EMC_RESTORE_DIFERENTS_CRCS	
02	ERROR_EMC_BACKUP_DIFERENTS_CRCS	
03	ERROR_3	
04	ERROR_ERC_WRITE_INDEX_OVERFLOW	
05	ERROR_ERC_READ_INDEX_OVERFLOW	
06	ERROR_6	
07	ERROR_7	
08	ERROR_CAN_RX_OVERFLOW	
09	ERROR_CAN_RX_INVALID_MSG	
10	ERROR_CAN_RX_XTD_FRAME	
11	ERROR_CAN_RX_RTR_FRAME	
12	ERROR_CAN_RX_DBL_BUFFERED	
13	ERROR_13	
14	ERROR_14	
15	ERROR_15	
16	EDR_EE_SPI_Write_WREN_COLLISION	
17	EDR_EE_SPI_Write_WRITE_COLLISION	
18	EDR_EE_SPI_Write_READ_COLLISION	
19	EDR_EE_SPI_Write_RDSR_COLLISION	
20	EDR_EE_SPI_Write_MSB_COLLISION	
21	EDR_EE_SPI_Write_LSB_COLLISION	
22	EDR_EE_SPI_Write_BYTE_COLLISION	
23	EDR_EE_SPI_Write_BUFFER_COLLISION	
24	EDR_EE_SPI_Write_PAGE_COLLISION	

Table: Dispositions des flags d'erreur sur la variable _error_status

Pour pouvoir activer ces bits d'erreur une **fonction publique** EDR_PARSER_DEBUGGER_SetErrorStaus est ajoute dans le PARSER et pourra être utilisé par les différents contrôleurs de mémoire et de communication CAN et SPI. Pour activer un type d'erreur il suffira de le passer une valeur préalablement déclare avec la directive #define et qui correspond avec le bit que on désire activer.

```
void EDR_PARSER_DEBUGGER_SetErrorStaus(unsigned long error_type){
   // Active a bit flag of the 32 bits variable  according to the error_type.
    _error_status = _error_status | error_type;
}
```

Finalement on ajoutera deux **fonctions privées** qui font partie de la liste de commandes acceptes par la requête Admin_Query.

1.2.2.1.1 Connaitre l'statut de la variable qui contient les flags d'erreur (GET_ERROR_STATUS).

Tout simplement retourne la valeur de la variable _error_status.

```
unsigned long EDR_PARSER_DEBUGGER_GetErrorStaus(void) { return _error_status;}
```

1.2.2.1.2 Réinitialiser variable qui contiens les flags d'erreur(RESET_ERROR_STATUS)

Elle va mettre à zéro la valeur de la variable _error_status.

```
void EDR_PARSER_DEBUGGER_ResetErrorStaus(void) { _error_status = 0;}
```

1.2.3 Fonctions ajoutes dans le Contrôleur de MEMOIRE

Comme on a vu précédemment certains codes d'opération seront traites par le contrôleur de mémoire (EDR_MEMORY_CTR). On décrit à continuation une série de fonctions gérées par ce contrôleur qui vont nous permettre de faciliter les essais.

1.2.3.1.1 Effacer une banque dans l'EEPROM externe (EEPROM_CLEAR_BANK).

Une fois qu'on a procédé à travailler avec l'EDR et on a réalisé des lectures et écritures l'état de la mémoire l'eeprom est modifié par rapport à l'état initial. Pour pouvoir réaliser des nouveaux tests en partant de un état connu il est bien outil de compter avec une fonction qui permet de **remettre la mémoire eeprom a l'état initial** (comme si elle contenait des valeurs après fabrication ou chaque cellule mémoire est rempli a «1», **0xFF** dans le cas d'un byte. Pour cela on ajoute la function:

```
unsigned long EDR_MEMORY_DEBUGGER_Clear_BANK(short aBank_Active) {..}
```

Cette fonction accepte comme paramètre un indicateur du banc qu'on veut effacer sur l'eeprom. Et a continuation réalise les actions suivantes :

- Efface toute les **Pages de Données** du banc sélectionne, et calcule la valeur CRC de toutes ces pages initialises a 0xFF.
- Sur la **Page de Control** du banc sélectionné registrera initialement deux paramètres consécutifs:
  - Le premier, est la valeur du **CRC Calcule** qui vient d'être obtenu pour les pages
  - Le seconde, ce la variable **Zone_Memoire** qui prendre la même valeur que l'identificateur du banque sélectionné reçu comme paramètre.

1.2.3.1.2 Restaurer les pages de donnes de la mémoire EEPROM vers la RAM (MEMORY_RESTORE_RAM)

Afin de pouvoir réaliser des cycles de test sans nécessite d'arrêter le debugger ou couper l'alimentation de la carte, une pétition Admin_Query avec la commande **MEMORY_RESTORE_RAM** va permettre de réaliser exactement la même fonction que est réalisé ou redémarre de la carte.

EDR_MEMORY_RESTORE_RAM() , elle se trouve déjà crée et explique préalablement dans le contrôleur de mémoire.

1.2.3.1.3 Connaitre le banc EEPROM utilise dans la restauration (GET_BANK_ACTIVE)

Ce fonction c'est l'équivalent de un getter pour la variable **_bank_Active** qui contiendra la valeur du banque de mémoire eeprom utilise lors de l'exécution de la fonction EDR_MEMORY_RESTORE_RAM().

```
unsigned char  EDR_MEMORY_DEBUGGER_GET_BANK_ACTIVE() { return _bank_Active;}
```

1.2.3.1.4 Connaitre les valeurs des CRCs sauvegardes dans les pages de control de l'EEPROM(GET_BANK_CRC)

A un moment donné, on pourra consulter les valeurs registres dans les deux Pages de Control de chaque banque de l'eeprom registres dans opérations précédents lors de l'exécution de la requête Recodring_Finish.

```
unsigned long  EDR_MEMORY_DEBUGGER_GET_BANK_CRC(unsigned char aBank_Active) {...}
```

Les compteurs d'événements

A niveau interne, on va ajouter des nouvelles fonctions de débogage qui vont permettre au programmeur de gérer deux compteurs d'évènements :

- **Recording_Finish**: Le premier indique le numéro de requetés «Recording_finish» reçues.
- **Recording_OK**: Le second le numéro de sauvegardes de la RAM vers la mémoire EEPROM dans le bloc sélectionne qui se sont réalisés avec succès.

Ces compteurs seront sauvegardes dans l'EEPROM dans la page de control de chaque bloc et seront places à continuation des deux valeurs métiers déjà présents qui sont le CRC calcule et la variable «Zone de Mémoire»

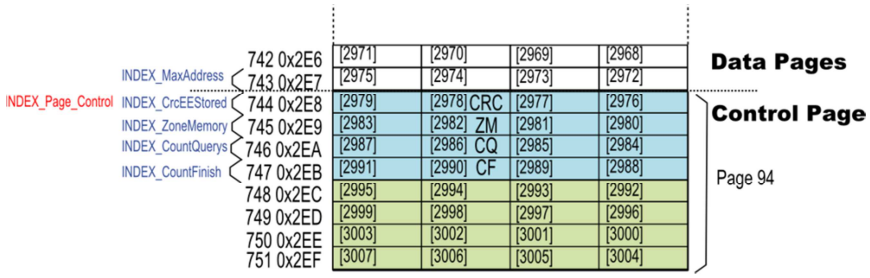


Figure: Placement des compteurs d'évènement sur la Page de Contrôle (Utilisant 93+1 Pages).

La différence entre ces deux compteurs permet de déduire le nombre de sauvegardes qui s'ont produit avec erreur et peut être une indication de que

- la mémoire EEPROM commence à présenter des **défaillances d'écriture/lecture**.
- il s'a produit une **coupure d'alimentation**, entre le moment de recevoir la requête de «Recording_finish» et la fin de la sauvegarde après la quel on incrément et on garde le compteur dans la page de control de l'EEPROM

1.2.3.1.5 Connaitre le nombre de pétitions Recording_Finish reçues (GET_COUNT_REC_QUERYS)

La valeur de **Recording_Finish** est incrémente juste après la réception de une trame Recording_Finish, et une fois incrémente la page de control es sauvegarde immédiatement dans l'EPROM.

La commande GET_COUNT_REC_QUERYS renvoi une trame Admin_Reply avec la valeur _t_value.countQuerys.

1.2.3.1.6 Connaitre le nombre d'enregistrements valides sur l'EEPROM (GET_COUNT_REC_OK)

La valeur de **Recording_OK** est incrémente après avoir sauvegarde la RAM dans l'EEPROM et a la comparution des CRCs de la RAM et de les valeurs relus de la EEPROM sont coïncidents indiçant que la sauvegarde est correcte.

La commande GET_COUNT_REC_OK renvoi une trame Admin_Reply avec la valeur _t_value.countFinish.

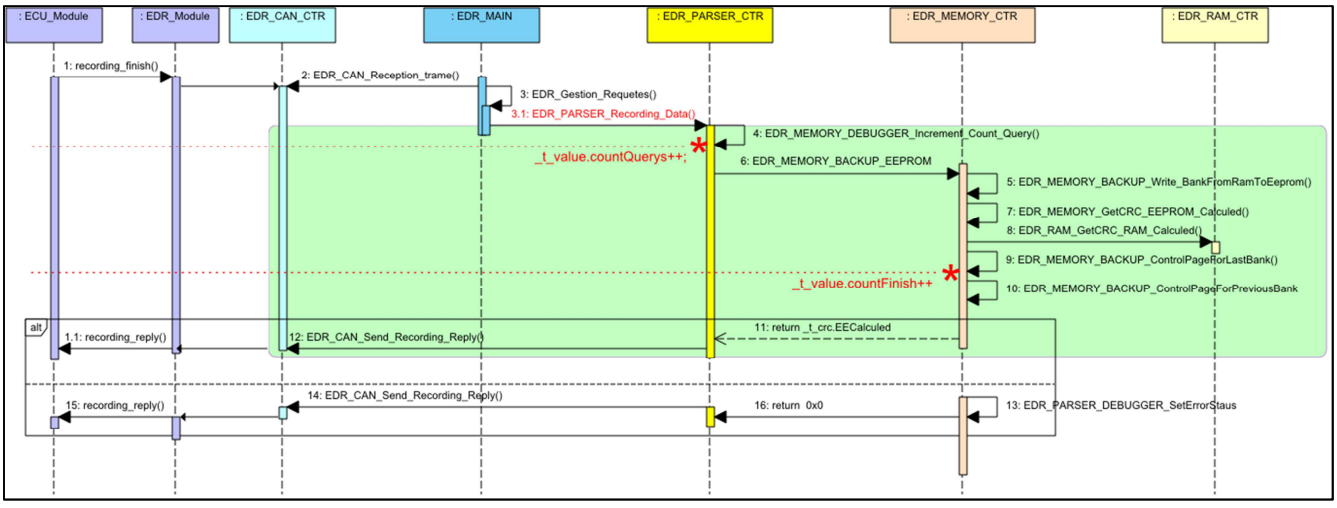


Figure: Placement des incréments des compteurs dans la séquence de Recording_Finish

2 Réalisation de tests unitaires pour validation des fonctions

Une fois réalisée les fonctions propres au métier EDR plus les fonctions additionnelles pour aider dans le débogage des fonctions métier, on va finaliser le développement par la création des fonctions qui vont nous permettre de réaliser des **tests unitaires** et on verra dans le cours de la réalisation des tests suivantes comme procède a la utilisation de ces fonctions.

Dans notre application on utilise une série de contrôleurs à différents niveaux, (Contrôleur de la RAM, de l'EPPROM, de la communication SPI, …). Ces contrôleurs s'appuient sur des **libraires de plus bas niveau** sur lesquelles on ne réalisera pas de tests directs, puisque indirectement si par exemple une libraire de communication présente un problème ceci sera détecté par les tests unitaires réalisés dans son contrôleur.

Pour chacun des principaux contrôleurs on va donc créer un fichier de Test avec toute une série de Test unitaires.

Un test unitaire va se réaliser sur une fonction C. On fera appel à l'intérieur a des fonctions métiers crées pour l'EDR que on veut vérifier, cette vérification on la réalise en trois étapes:

1. On procèdera à l'appel des **fonctions d'écriture**: On imposera des valeurs d'entrée connus à ces fonctions métiers (ex: sollicitant d'écrire une certaine valeur a une certain position mémoire)
2. On procèdera à l'appel des **fonctions de lecture**: Et ces fonctions nous donneront des valeurs en retour.
3. On réalise la **comparaison des résultats**. En vérifiant que les valeurs de retour reçus son identiques a les valeurs d'entrée connus.

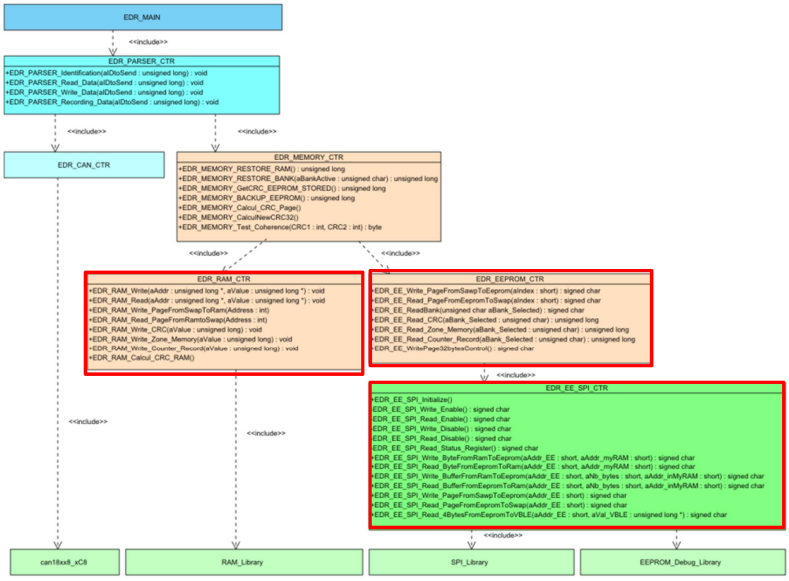


Figure: Fonctions à tester dans les principaux contrôleurs

Dans notre projet on ajoutera un dossier sous le répertoire **src/Test** pour écrire ces fonctions de test. A l'intérieur on se trouvera les sub-directoires **Memory** pour écrire les tests unitaires des contrôleurs RAM, EEPROM et communication SPI) et **Protocol** (pour écrire les tests unitaires du Parser).

Ces test pourront ce valider:

- A travers des **messages envoyés par le port série** en utilisant les fonctions crées à ce propos. Cette option est plus rapide et permet de **vérifier rapidement les modifications dans le code des contrôleurs**.
- A travers de MPLABX en utilisant des breakpoints et les fonctions de **debug pas à pas**. Cet option pourra être utilise quand on n'aura pas de sorties disponibles pour la transmission série TX.

2.1 Auto validation des fonctions du Contrôleur RAM.

Objectifs du test

Ces fonctions décrites dans le fichier **src/Test/Memory/EDR_RAM_CTR_Test.c/.h** vont permettre de valider les fonctions métiers programmées pour la lecture/écriture des valeurs sur la mémoire RAM. Les tests vont valider:

- L'écriture/lecture d'une valeur de **32 bits** sur différents pages de mémoire.
- L'écriture/lecture d'une valeur de 32 bits sur la page de control qui devra donner une erreur (on indique que l'erreur est attendue en ajoutant à la fin de la ligne de log «**…E:**»)
- L'écriture/lecture d'une **page** de la SWAP vers les différents pages de la RAM y compris la page de control, et vice-versa.
- L'écriture/lecture d'une valeur de 32 bits depuis une **variable** ver la Page de control, et vice-versa.
- Le calcul de la valeur du CRC pour une RAM remplie des bytes avec la valeur 0xFF.

Moyens d'essais et configuration hardware

Pour la réalisation de ce test on va utiliser les éléments décrites dans le schéma montre dans le point décrit précédemment «**Gestion de transmission de données série pour le débogage**» qui contient:

- Le programmateur debugger PICKit3 (Avec l'adaptateur ICSP AC164110 6-pins femelles /RJ-11)
- Une carte d'évaluation PICDEM CAN-LIN3 ;
- La carte EDR connecté au debugger PICKit3 et au port série du PC a travers du MAX232N de la carte d'évaluation PICDEM
- Le logiciel MPLABX avec le compilateur xC8.
- Le logiciel RealTerm configure pour la communication série à 9600 bauds. ;

Procédure utilisée

1. Sur **RallTerm** configurer le port de communication câblé pour fonctionner à une vitesse de **9600 baud**.
2. Dans le fichier EDR_Test.h on ajoute un define pour activer les fonctions de Test qui vont permettre de envoyer des lignes de log a travers du port série.
   ```
   #define TEST_LOG_SERIAL
   ```
3. Dans la fonction main on vérifie que la fonction est bien appelle dans le test conditionnel du define:
   ```
   #ifdef TEST_LOG_SERIAL
       Test_RAM_CTR();
   #endif
   ```
4. On allume la carte EDR, on lance l'application en mode debugge avec MPLABx. et on observe les résultats dans l'application **RealTerm**.

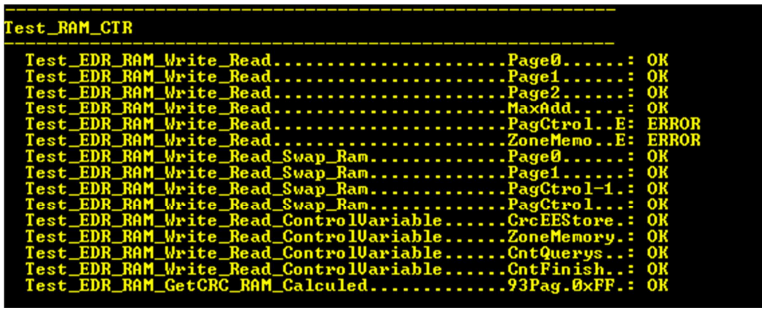


Figure: Messages de log pour chacun des tests unitaires du contrôleur RAM.

2.2 Auto validation des fonctions du Contrôleur SPI.

Objectifs du test

Ces fonctions décrites dans le fichier **src/Test/Memory/ EDR_EE_SPI_CTR_Test.c/.h** vont permettre de valider les fonctions métiers programmées pour l'envoi/réception des commandes à travers le port SPI pour réaliser des écriture/lecture sur la mémoire EEPROM externe. Les tests vont valider:

- **Les connexions électriques réalisées dans la carte EDR entre le microcontrôleur et la mémoire EEPROM externe.**
- L'écriture/lecture d'une valeur de **un byte** transfère de la RAM vers l'EEPROM (depuis n'importe quel adresse de la RAM vers n'importe quel adresse de l'EEPROM) et vice-versa.
- L'écriture/lecture d'une valeur de **un buffer** de bytes transfère de la RAM vers l'EEPROM. La taille du buffer doit être une valeur inferieur à la taille d'une page, c'est-à-dire 32 bytes. Si le nombre de bytes indique dans le buffer dépasse les 32 la fonction la comparaison de résultats donnera une erreur attendue («**…E:**»). Ceci est dû au fait que dans une écriture dans l'eeprom de plus de 32 bits l'eeprom réalise une écriture cyclique consistant à écrire les bytes suivantes sur le début de l'adresse indique (ex: Si on désire écrire une série de 40 nombres dans l'adresse 20, après avoir écrit les premiers 32 correctement [0@20, 1@21, 2@22,…,31@51] les suivantes seront gardés à partir de l'adresse 20 [32@20, 33@21, 34@22,…].
- L'écriture/lecture d'une **page** de la SWAP vers les différents pages de l'EEPROM y compris la page de control, et vice-versa (Différents test seront réalisés autant dans le banc logique 0 que dans le banc logique 1.).
- L'écriture/lecture d'une valeur de 32 bits depuis une **variable** ver les pages de control situes dans chacun des bancs, et vice-versa.

Moyens d'essais et configuration hardware

Pour la réalisation de ce test on va utiliser la même configuration décrite dans le test précèdent 2.1.

Procédure utilisée

Même séquence que dans le test précèdent sauf que la fonction appelé est `Test_EDR_EE_SPI_CTR();`.

```
Test_EDR_EE_SPI_CTR
-------------------------------------------------------
  Test_EDR_EE_SPI_Write_Read_Ram_Eeprom.......E0-R0.....: OK
  Test_EDR_EE_SPI_Write_Read_Ram_Eeprom.......E1-R1.....: OK
  Test_EDR_EE_SPI_Write_Read_Ram_Eeprom.......E7-R8.....: OK
  Test_EDR_EE_SPI_Write_Read_Ram_Eeprom.......E15-R16....: OK
  Test_EDR_EE_SPI_Write_Read_Ram_Eeprom.......MaxAdd.....: OK
  Test_EDR_EE_SPI_Write_Read_Ram_Eeprom.......PagCtrol...: OK
  Test_EDR_EE_SPI_Write_Read_Ram_Eeprom.......ZoneMemory.: OK
  Test_EDR_EE_SPI_Write_Read_Buffer_Ram_Eeprom.E0R0B4.....: OK
  Test_EDR_EE_SPI_Write_Read_Buffer_Ram_Eeprom.E0R0B8.....: OK
  Test_EDR_EE_SPI_Write_Read_Buffer_Ram_Eeprom.E0R0B16....: OK
  Test_EDR_EE_SPI_Write_Read_Buffer_Ram_Eeprom.E0R0B30....: OK
  Test_EDR_EE_SPI_Write_Read_Buffer_Ram_Eeprom.E0R0B31....: OK
  Test_EDR_EE_SPI_Write_Read_Buffer_Ram_Eeprom.E0R0U32....: OK
  Test_EDR_EE_SPI_Write_Read_Buffer_Ram_Eeprom.E0R0B33...E: ERROR
  Test_EDR_EE_SPI_Write_Read_Buffer_Ram_Eeprom.E16R16B16..: OK
  Test_EDR_EE_SPI_Write_Read_Buffer_Ram_Eeprom.E8R12B4....: OK
  Test_EDR_EE_SPI_Write_Read_Buffer_Ram_Eeprom.E12R20B8...: OK
  Test_EDR_EE_SPI_Write_Read_Buffer_Ram_Eeprom.E32R64B16..: OK
  Test_EDR_EE_SPI_Write_Read_Page_Sawp_Eeprom..B1P1@0.....: OK
  Test_EDR_EE_SPI_Write_Read_Page_Sawp_Eeprom..B1P2@32....: OK
  Test_EDR_EE_SPI_Write_Read_Page_Sawp_Eeprom..B1P3@64....: OK
  Test_EDR_EE_SPI_Write_Read_Page_Sawp_Eeprom..B1P4@96....: OK
  Test_EDR_EE_SPI_Write_Read_Page_Sawp_Eeprom..B1LastDatPg: OK
  Test_EDR_EE_SPI_Write_Read_Page_Sawp_Eeprom..B1ControlPg: OK
  Test_EDR_EE_SPI_Write_Read_Page_Sawp_Eeprom..B2P1.......: OK
  Test_EDR_EE_SPI_Write_Read_Page_Sawp_Eeprom..B2P2.......: OK
  Test_EDR_EE_SPI_Read_4BytesFromEepromToVBLE..CrcEEStored: OK
  Test_EDR_EE_SPI_Read_4BytesFromEepromToVBLE..ZoneMemo...: OK
  Test_EDR_EE_SPI_Read_4BytesFromEepromToVBLE..countQuerys: OK
  Test_EDR_EE_SPI_Read_4BytesFromEepromToVBLE..countFinish: OK
```

Figure: Messages de log pour chacun des tests unitaires du contrôleur SPI de la mémoire EEPROM.

2.3 Auto validation des fonctions du Contrôleur EEPROM

Objectifs du test

Ces fonctions décrites dans le fichier **src/Test/Memory/EDR_EEPROM_CTR_Test.c/.h** vont permettre de valider les fonctions métiers programmées pour la lecture/écriture des valeurs sur la mémoire EEPROM. Ces fonctions vont s'appuyer sus les fonctions de communication SPI testes précédemment et elles vont ajouter un degré d'abstraction facilitant la codification des séquences de restauration de mémoire de EEPROM vers RAM et backup de RAM vers EEPROM. Les tests vont valider:

- L'écriture/lecture d'une **page** de la SWAP vers les différents pages de l'EEPROM y compris la page de control, et vice-versa (Diffèrent test seront réalisés autant dans le banc logique 0 BANK0 que dans le banc logique 1 BANK1).
- La lecture de un **banc complet** de l'EEPROM vers la RAM. Cet équivalent à la lecture de un buffer de bytes de l'EEPROM vers la RAM ou la taille du buffer est la taille de tous les bytes places dans les pages de données de un banc sélectionne sur l'EEPROM (BANK0 ou BANK1).
- La lecture des **variables places dans les pages de contrôle** de l'EEPROM des deux bancs vers de variables temporaires places dans la RAM.
- L'écriture de la page de contrôle place dans la RAM vers une des pages de contrôle de l'EEPROM selon la valeur du banc sélectionne (BANK0 ou BANK1).

Moyens d'essais et configuration hardware

Pour la réalisation de ce test on va utiliser la même configuration décrite dans le test précèdent 2.1.

Procédure utilisée

Même séquence que dans le test précèdent sauf que la fonction appelé est `Test_EDR_EEPROM_CTR();`.

```
-------------------------------------------------------
Test_EDR_EEPROM_CTR
-------------------------------------------------------
  EDR_EE_Write_Read_Page_Sawp_Eeprom...........B0P1.......: OK
  EDR_EE_Write_Read_Page_Sawp_Eeprom...........B0P2.......: OK
  EDR_EE_Write_Read_Page_Sawp_Eeprom...........B0P3.......: OK
  EDR_EE_Write_Read_Page_Sawp_Eeprom...........B0Control..: OK
  EDR_EE_Write_Read_Page_Sawp_Eeprom...........B1P1.......: OK
  EDR_EE_Write_Read_Page_Sawp_Eeprom...........B1P2.......: OK
  EDR_EE_Write_Read_Page_Sawp_Eeprom...........B1P3.......: OK
  EDR_EE_Write_Read_Page_Sawp_Eeprom...........B1Control..: OK
  EDR_EE_Read_BankFromEepromToRam..............BANK0......: OK
  EDR_EE_Read_BankFromEepromToRam..............BANK1......: OK
  EDR_EE_Read_Variables........................BANK0......: OK
  EDR_EE_Read_Variables........................BANK1......: OK
  EDR_EE_Write_Page32bytesControl..............BANK0......: OK
  EDR_EE_Write_Page32bytesControl..............BANK1......: OK
```

Figure: Messages de log pour chacun des tests unitaires du contrôleur de la mémoire EEPROM.

3 Réalisation de tests externes à l'aide de un CAN Bus Analyzer

Une fois valide le comportement des fonctions réalisant les spécifications décrites pour le **fonctionnement interne** de l'EDR on va procéder à réaliser les premiers tests de communication CAN avec un dispositif externe simulant le comportement que l'ECU. On dispose d'un protocole simple base sur l'envoi de 4 types de trames pour consulter, modifier et sauvegarder l'information des paramètres moteurs dans l'EDR, plus une source d'alimentation externe que va nous permettre de placer l'EDR en mode «**opérationnel**» puis en mode «**Hors-tension**». Apres une nouvelle initialisation de la carte on doit vérifier que notre information a était bien registre et l'intégrité des données est maintenue.

Moyens d'essais et configuration hardware

Pour la réalisation de la prochaine série de tests on va utiliser les éléments décrits dans le schéma suivant qui contient:

- Le programmateur debugger PICKit3 (Avec l'adaptateur ICSP AC164110 6-pins femelles /RJ-11)
- Une source d'alimentation stabilisée externe fixe à +10v.
- La carte EDR connecté au debugger PICKit3 et au port série du PC a travers du MAX232N de la carte d'évaluation PICDEM
- Le logiciel MPLABX avec le compilateur xC8.
- L'adaptateur "**PC USB Port to High-speed CAN**" de **PEAK**-System Technik GmbH (www.peak-system.com)
- Le logiciel fourni avec l'adaptateur CAN: PcanViewUsb.exe (Version 2.5.0.75) configure à une vitesse de 500kbit/s

Pour la réalisation de ces test il est nécessaire de connecter l'EDR vers l'adaptateur USB-CAN du PC à travers d'un bus CAN. Pour cela on soude un **câble tressée** de 2 fils sur les sorties (CAN_L, CAN_H) du transceiver CAN de l'EDR qui sera après liée à l'adaptateur USB-CAN comme on montre dans le schéma suivant:

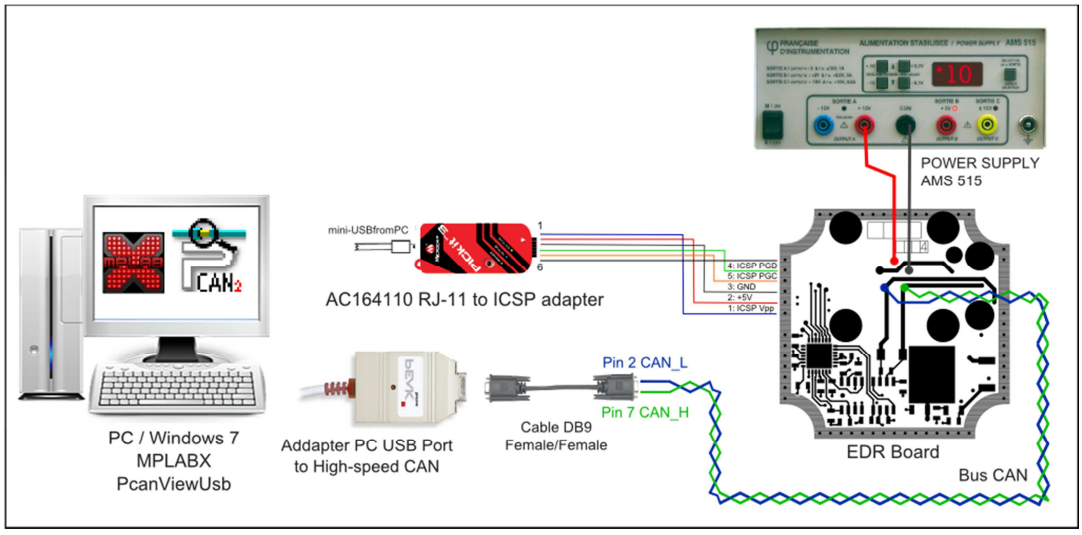


Figure: Connexions de la carte EDR avec le PC et l'alimentation stabilisée.

3.1 Test pour valider la communication CAN 2.0A EXI_EDR_3

Pour valider la communication CAN avec l'interface EDR on dispose initialement de deux dispositifs analyseur de bus CAN libres avec de logiciels gratuits:

- Un du fabricant **Microchip**, avec le logiciel **Microchip CAN Bus Analyzer** Version 2.0
  http://www.microchip.com/stellent/idcplg?IdcService=SS_GET_PAGE&nodeId=1406&dDocName=en546534
- Un autre du fabricant **Peak-System**, avec le logiciel **PCAN-View** version 3.2.3.221
  http://www.peak-system.com/Product-Details.49.0.html?&L=2&tx_commerce_pi1%5BcatUid%5D=10&tx_commerce_pi1%5BshowUid%5D=32

Le logiciel de Microchip étant un peu pauvre en fonctionnalités offertes, on utilisera pour la réalisation de ces tests celui de **Peak** puisque entre autres avantages il permet de:

- **Définir un nombre indéfini de messages** de transmission (Celui de Microchip est limite)
- **Registrer la configuration de travail** dans un fichier avec la vitesse de communication et les messages de transmission qu'on pourra récupérer dans d'autres sessions.

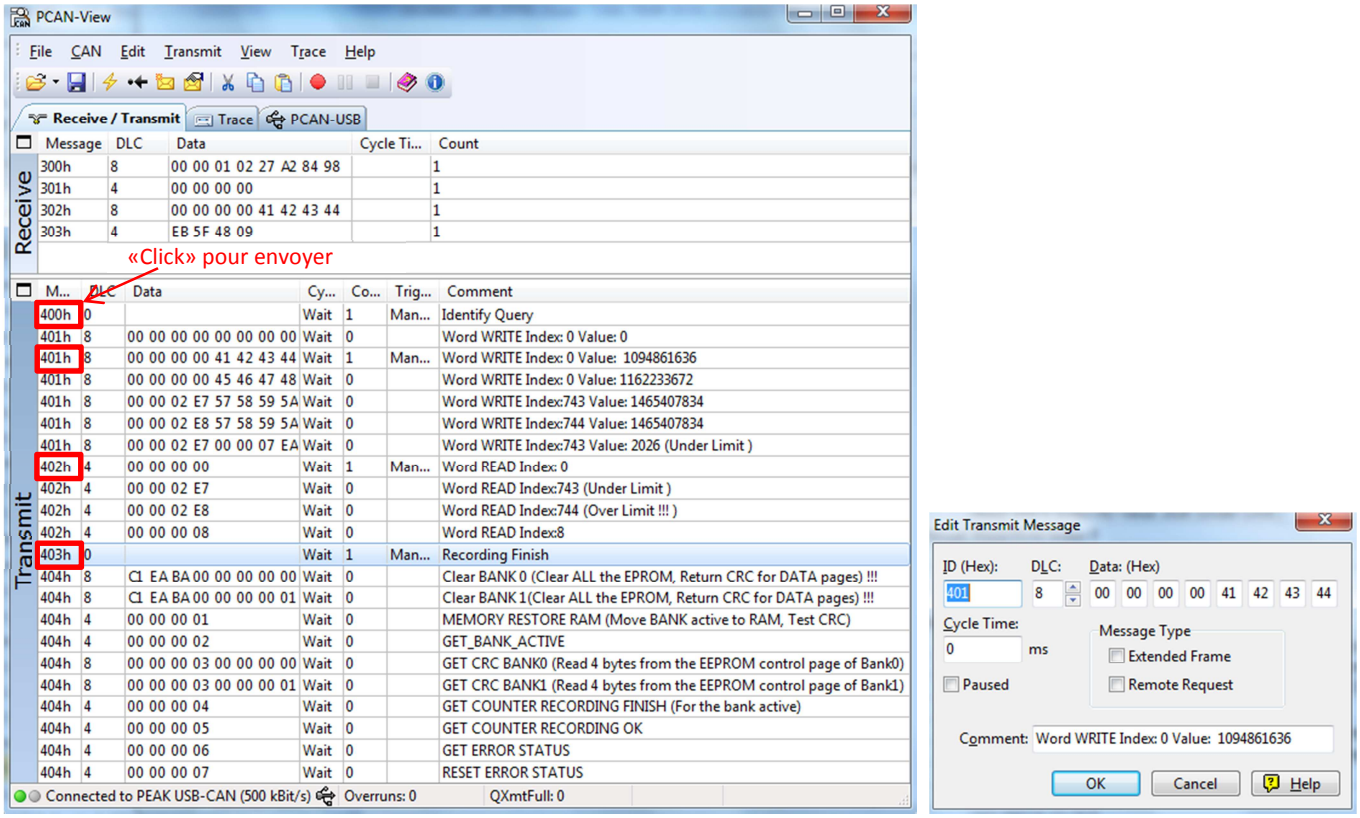


Figure: Capture d'écran de la configuration de PCAN-View registré dans le fichier EDR_TEST_CAN_ADMIN.xmt

Procédure utilisée

1. Lancer le logiciel PCAN-View et établir une connexion à 500 kbit/sec
2. Ajouter toute une série de messages de transmission comme décrit la figure précèdent.
3. Envoyer initialement pour tester les 4 types de trames du protocole EDR avec les IDs établies:
   1. **0x400** Pour envoyer un Identify_Read, on doit recevoir la trame Identify_Reply
   2. **0x401** Pour envoyer un Word_write pour écrire *les caractères «ABCD» dans l'adresse 0x0.*
   3. **0x402** Pour envoyer un Word_read pour lire la valeur registre dans l'adresse 0x0. Si dans la trame de réponse on obtient la valeur écrite précédemment cela permet de **valider les fonctions d'écriture/lecture**.
   4. **0x403** Pour envoyer un Recording_Finish, pour solliciter le backup de la RAM vers l'EEPROM.

3.2 Test de la librairie SPI (Interface pour la mémoire EEPROM externe)

3.2.1 Test des fonctions de la librairie SPI bas niveau.

Objectifs du test

A continuation on va tester à l'oscilloscope le comportement des signaux qui forment le bus SPI accompagné de l'activation/désactivation de la ligne de CS Chip Select lors des étapes nécessaires pour écrire un byte dans l'EEPROM

Moyens d'essais et configuration hardware

A partir de le schème décrit dans le chapitre «Codification des fonctions gérant la communication SPI» qu'utilise une mémoire EEPROM externe on va connecter 4 sondes de l'oscilloscope vers les signaux:

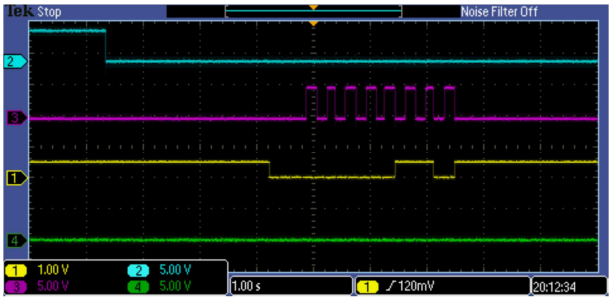
- (Blue) **CS** : Chip Select
- (Violet) **SCK** : Serial Clock
- (Jaune) **MOSI** : Master Output Serial Input
- (Vert) **MISO** : Master Input Serial Output

Procédure utilisée

En mode debug on réalise une phase d'écriture de la valeur 0x48 dans l'adresse 0x8, avec les actions suivantes:

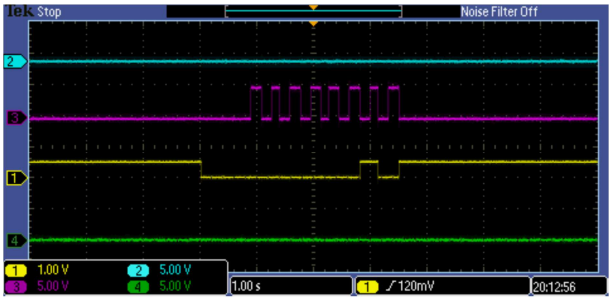
1. EEPROM_CS = 0; // Chip select pour activation bus SPI
2. EEPROM_WP = 1; // On désactive la protection en écriture sur l'EEPROM
3. Puis on envoie la séquence de bytes a la mémoire a travers du bus SPI avec les instructions et données:

3



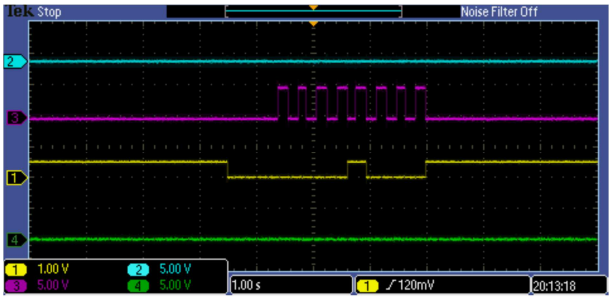
WriteSPI(SPI_WREN); // 0x6 write enable latch

4



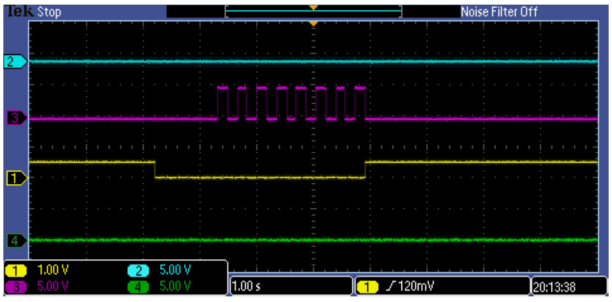
WriteSPI(SPI_WRITE); // 0x2 write data to memory

5



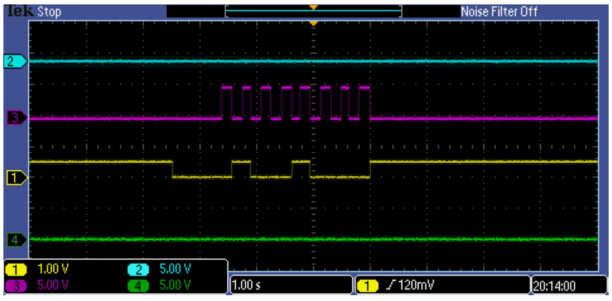
WriteSPI(aAddr_EE); // 0x8 Address LSO

6



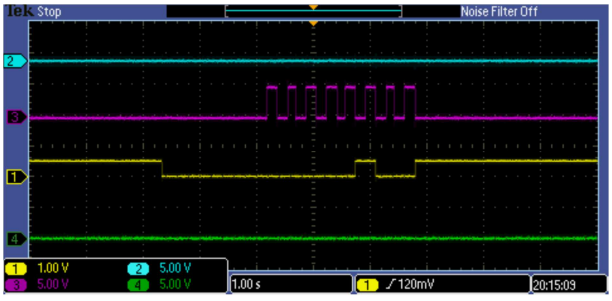
WriteSPI(aAddr_EE>>8); // 0x0 Address MSO

7



WriteSPI(prtBYTEinMyRAM[i]); // 0x48 Character 'H'

8



WriteSPI(SPI_WRDI); // 0x4 reset the write enable latch

3.2.2 Test des fonctions du contrôleur SPI.

Objectifs du test

A niveau du contrôleur on veut faire appeler à deux fonctions pour réaliser l'écriture/lecture de une valeur donne sur un adresse mémoire connue. Ce sont ces fonctions qui vont réaliser la séquence d'écriture des bytes dans le bus SPI en utilisent les fonctions de bas niveau **WriteSPI/ReadSPI** de la librairie SPI décrite précédemment.

Moyens d'essais et configuration hardware

On procède avec la mémé configuration que celle décrite dans le test précèdent. A partir de le schème décrit dans le chapitre «Codification des fonctions gérant la communication SPI» qu'utilise une mémoire EEPROM externe on va connecter 4 sondes de l'oscilloscope vers les signaux:

- (Blue) **CS** : Chip Select
- (Violet) **SCK** : Serial Clock
- (Jaune) **MOSI** : Master Output Serial Input
- (Vert) **MISO** : Master Input Serial Output

Procédure utilisée

Les fonctions du contrôleur destines au **transfert d'information** vont transférer principalement des données de la RAM ver l'EEPROM. Dans le cas des fonctions qu'on veut analyser, celles-ci vont concrètement transférer un byte depuis un tableau dans la RAM vers l'EEPROM et vice-versa.

1. Pour cela il est nécessaire d'écrire dans la RAM préalablement l'information que on veut registrer à continuation dans l'EEPROM (dans notre exemple addr_myRAM =0x0, test1_Value = 0x48)
   RAM_Write(&addr_myRAM, &test1_Value);
2. A continuation on réalise l'écriture de un byte avec la valeur 0x48 dans l'address EEPROM 0x8.
   EDR_EE_SPI_Write_ByteFromRamToEeprom(addr_EEPROM, addr_myRAM*4);
3. Et on finalise en réalisant une lecture de l'adresse EEPROM 0x8 utilise dans l'écriture précèdent pour vérifier que on est arrivé à registrer et lire correctement ce valeur.
   EDR_EE_SPI_Read_ByteFromEepromToRam(addr_EEPROM, addr_myRAM*4);

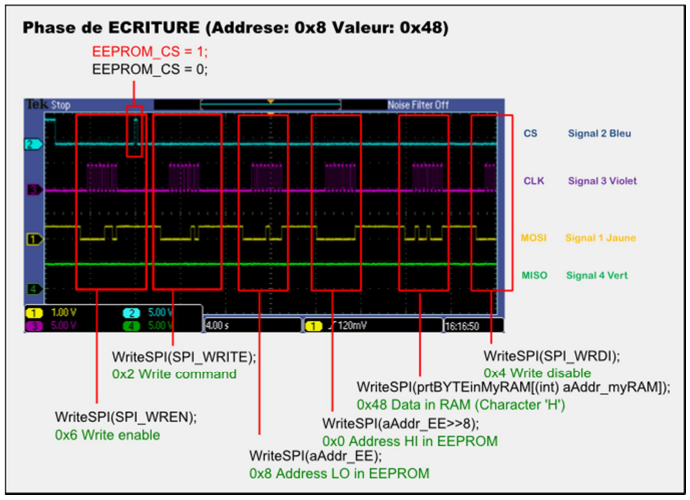


Figure: Séquence d'écriture a l'oscilloscope.

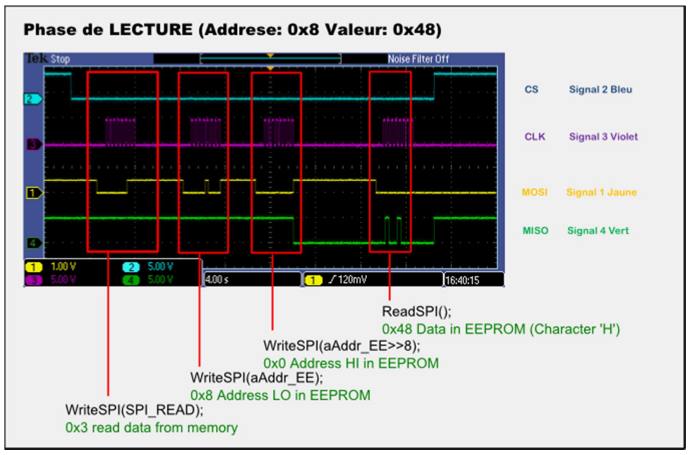


Figure: Séquence de lecture a l'oscilloscope.

Dans la **séquence d'écriture** d'un byte, observer comme au moment d'envoyer l'instruction **WREN** Write Enable il est nécessaire de gérer correctement la désactivation du **CS** Chip Select pour après l'activer a nouveau pendant l'envoi de l'instruction WRITE et les bytes qui suivent a continuation.

Dans la **séquence de lecture** observer comme **l'EPROM active la ligne MISO** avec la valeur lue à l'adresse donnée.

3.3 Test du cas d'utilisation «Restaurer la mémoire» EXI_EDR_1

Objectifs du test

Le cas d'utilisation «Restaurer la mémoire» se produit au moment d'alimenter la carte EDR et une fois initialise toute les modules de communication. L'objectif initial de ce test et de **mesurer le temps** nécessaire pour réaliser la restauration complète de tous les données de la EEPROM vers la RAM. Cette restauration se réalise en plusieurs étapes et on désire mesurer aussi te temps de réalisation de chacune. Pour cela on utilisera l'oscilloscope et on ajoutera dans le code des lignes que vont activer et désactiver les pins disponibles et obtenir ainsi une mesure.

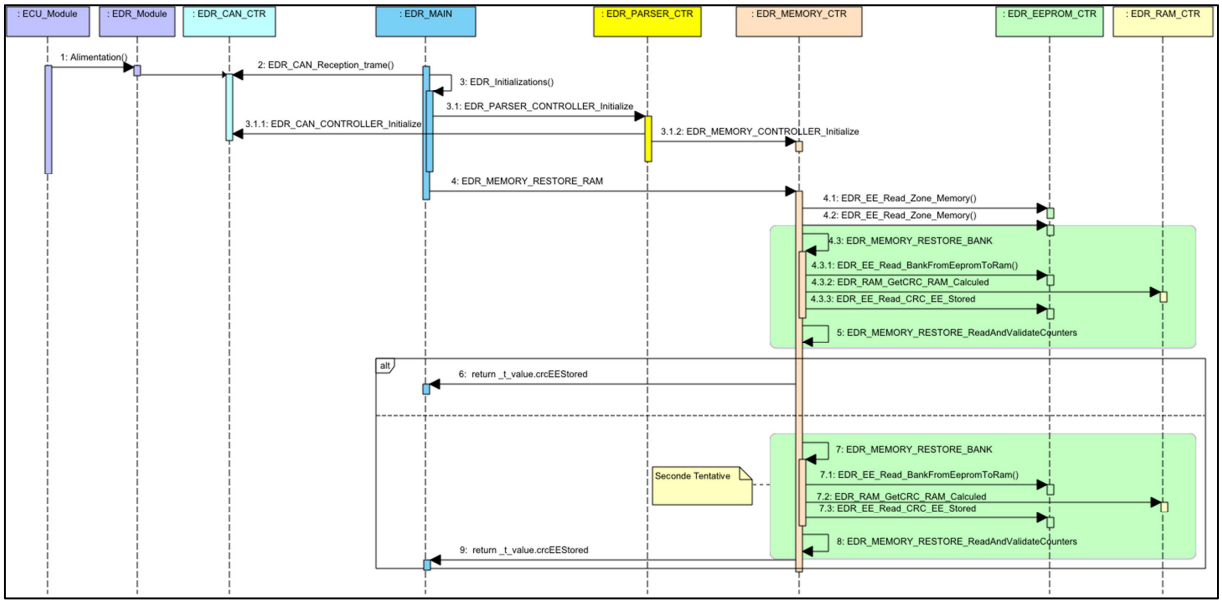


Figure : Diagramme de séquence du cas d'utilisation «Restaurer la mémoire»

Moyens d'essais et configuration hardware et software

Pour la réalisation de ce test, on va utiliser la même **configuration hardware** décrite au début de ce chapitre.

Pour ce qui concerne la **configuration software** on se placera dans le fichier EDR_MEMORY_CTR sur la fonction EDR_MEMORY_RESTORE_BANK () pour ajouter les lignes d'activation/désactivation des sorties:

- Le pin **RB4** (Jaune) pour mesurer le **Transfert des données EEPROM vers la RAM**

```
IO_DEBUGGER_Set_LEDs(&LATB,0x10);
    EDR_EE_Read_BankFromEepromToRam(aBankActive);
IO_DEBUGGER_Set_LEDs(&LATB,0x0);
```

- Le pin **RB5** (Blue) pour mesurer le **Calcul du CRC de la RAM**

```
IO_DEBUGGER_Set_LEDs(&LATB,0x20);
    _t_crc.RAMCalculed   = EDR_RAM_GetCRC_RAM_Calculed();
IO_DEBUGGER_Set_LEDs(&LATB,0x0);
```

Les actions finales du cas d'utilisation qui restent prennent un temps insignifiant à cote no considérable:

- Récupération du CRC registre dans l'EEPROM
- Test de Cohérence

Finalement pour mesurer le temps complet de tout le cas d'utilisation on se placera dans le fichier EDR_MAIN sur la fonction main() pour l'activation de une autre sortie

- Le pin **RA0** (Vert) pour mesurer le **temps total d'exécution de la restauration de mémoire.**

```
IO_DEBUGGER_Set_LEDs(&LATA,0x1);
    EDR_MEMORY_RESTORE_RAM();
IO_DEBUGGER_Set_LEDs(&LATA,0x0);
```

Procédure utilisée

1. Réaliser la configuration hardware décrite précédemment.
2. Inclure dans le code l'activation/désactivation des pins comme décrit précédemment.
3. Ajouter les points de mesure avec les sondes de l'oscilloscope sur les pins configures au niveau logiciel :
   - La sonde 1 (jaune) sur le pin **RB4**. (Mesure le temps de transfert de l'EEPROM vers la RAM)
   - La sonde 2 (bleu) sur le pin **RB5**. (Mesure le temps de calcul du CRC de la RAM)
   - La sonde 4 (vert) sur le pin **RA0**. (Mesure le temps total pour restaurer la mémoire).
4. Alimenter la carte EDR, redémarrer l'application et observer la valeur des signaux. A l'aide de l'oscilloscope on peut ajouter des fonctions de mesures pour les différents signaux et on peut lire ainsi la valeur de temps :
   - **224.1 msec** Temps de transfert de l'EEPROM vers la RAM.
   - **182.5 msec** Temps de calcul du CRC de la RAM.
   - **421.5 msec** Temps total pour restaurer la mémoire.

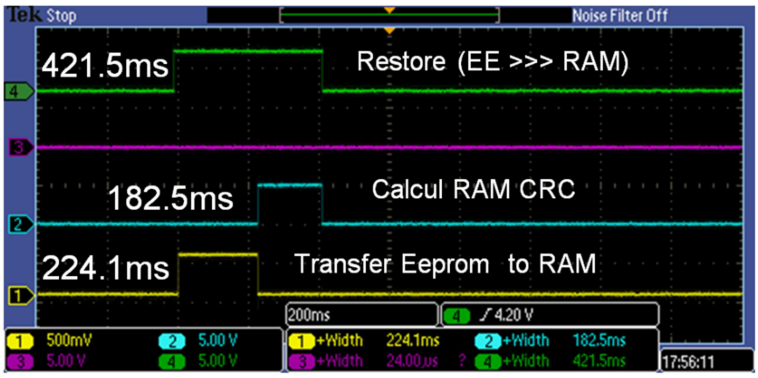


Figure : Capture d'écran de l'oscilloscope lors de la réalisation de la restauration de mémoire.

Comme on avait indiqué, on observe que les 2 fonctions prennent la plus part du temps de l'exécution et les actions finales (Récupération du CRC registre dans l'EEPROM et Test de Cohérence) prennent un temps insignifiant en comparaison.

421.5 msec – (224.1 msec + 182.5 msec) = 421.5 msec – 406.6 msec = **14,9 msec**

Ce test nous permet de obtenir la mesure de temps, mais il ne nous indique pas pour l'instant que la restauration de la mémoire est réalisé correctement et l'intégrité des données et maintenu. Cela on va réaliser dans les tests suivants en combinant les fonctions d'écriture, lecture et calcul des CRCs.

3.4 Test du cas d'utilisation «Solliciter information» EXI_EDR_7

Objectifs du test

Le cas d'utilisation «Solliciter Information » se produit lors de la réception de une trame CAN «Identify_Read» et au niveau interne l'EDR récupère les valeurs d'identification matériel et logiciel plus le CRC des donnes charges en mémoire RAM lors de la restauration précèdent.

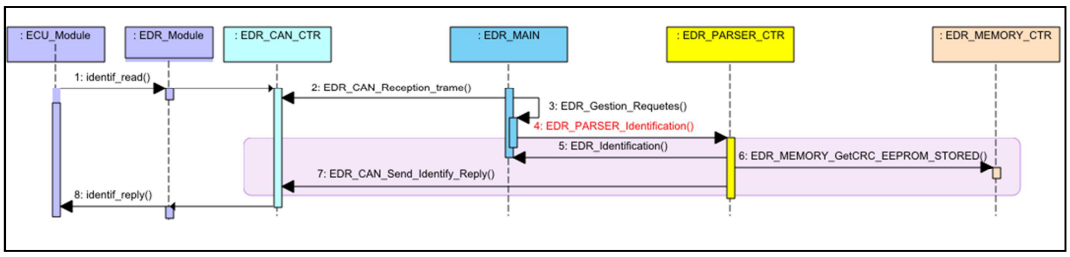


Figure : Diagramme de séquence du cas d'utilisation «Restaurer la mémoire»

Moyens d'essais et configuration hardware et software

Pour la réalisation de ce test, on va utiliser la même **configuration hardware** décrite au début de ce chapitre.

Pour ce qui concerne la **configuration software** on se placera dans le fichier EDR_PARSER_CTR sur la fonction EDR_PARSER_Identification () pour ajouter une ligne d'activation/désactivation de sortie:

- Le pin **RB4** (Vert) pour mesurer le **temps d'exécution de la commande «Identify_Query»**

```
void EDR_PARSER_Identification( unsigned long aIDtoSend){
IO_DEBUGGER_Set_LEDs(&LATB,0x10);
    // Recover the hardware and software identifiers
    EDR_MAIN_GetIdentification(&hW_VERSION, &sW_VERSION, &sW_REVISION);

    // Recover the CRC Stored in EEPROM at the initialization.
    _t_value.crcEEStored = EDR_MEMORY_GetCRC_EEPROM_STORED();

    // Send the identification reply whit the identifiers and CRC stored.
    EDR_CAN_Send_Identify_Reply(
            aIDtoSend,hW_VERSION,sW_VERSION,sW_REVISION,_t_value.crcEEStored);
IO_DEBUGGER_Set_LEDs(&LATB,0x0);
}
```

Procédure utilisée

1. Réaliser la configuration matériel et logiciel comme décrit précédemment.
2. Ajouter les points de mesure avec les sondes de l'oscilloscope sur les pins configures au niveau logiciel :
   - La sonde 4 (vert) sur le pin RB4. (Mesure le temps d'exécution de la commande «Identify_Query»).
3. Alimenter la carte EDR et observer la valeur des signaux. A l'aide de l'oscilloscope on peut ajouter des fonctions de mesures pour les différents signaux et on peut lire ainsi la valeur de temps :
   - **334.8 µsec** Temps d'exécution de la commande «Identify_Query».

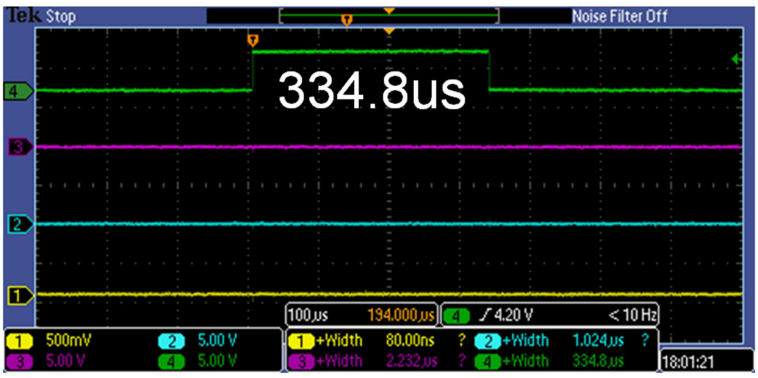


Figure : Capture d'écran de l'oscilloscope lors de la sollicitation de l'information.

3.5 Test du cas d'utilisation «Ecrire des données» EXI_EDR_4, 13 et 14

Objectifs du test

Le cas d'utilisation «Ecrire des donnes » se produit lors de la réception d'une trame CAN «Word_Write» et au niveau interne l'EDR décrite les valeurs reçus dans la trame pour réaliser l'écriture sur la RAM d'une valeur donnée dans une adresse donnée.

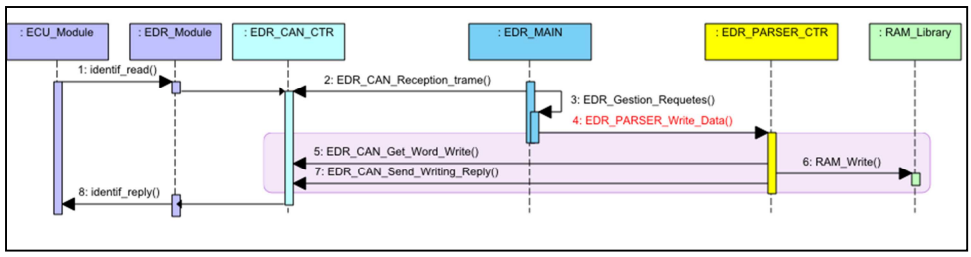


Figure : Diagramme de séquence du cas d'utilisation «Ecrire des données»

Moyens d'essais et configuration hardware et software

Pour la réalisation de ce test, on va utiliser la même **configuration hardware** décrite au début de ce chapitre.

Pour ce qui concerne la **configuration software** on se placera dans le fichier EDR_PARSER_CTR sur la fonction EDR_PARSER_Write_Data () pour ajouter une ligne d'activation/désactivation de sortie:

- Le pin **RB4** (Vert) pour mesurer le **temps d'exécution de la commande «Word_Write»**

```
void EDR_PARSER_Write_Data( unsigned long aIDtoSend){
IO_DEBUGGER_Set_LEDs(&LATB,0x10);
    // Unpack the CAN frame information to recover the write indicators.
    EDR_CAN_Get_Word_Write(&wordWriteIndex, &wordWrite);

    // Validate and write the value received at the index received
    EDR_RAM_Write(&wordWriteIndex, &wordWrite);

    // Send the writing reply whit the index received (or 0xBADC0DE if error)
    EDR_CAN_Send_Writing_Reply(aIDtoSend, wordWriteIndex);
IO_DEBUGGER_Set_LEDs(&LATB,0x0);
}
```

Procédure utilisée

1. Réaliser la configuration matériel et logiciel comme décrit précédemment.
2. Ajouter les points de mesure avec les sondes de l'oscilloscope sur les pins configures au niveau logiciel :
   - La sonde 4 (vert) sur le pin RB4. (Mesure le temps d'exécution de la commande «Word_Write»).
3. Alimenter la carte EDR et observer la valeur des signaux. A l'aide de l'oscilloscope on peut ajouter des fonctions de mesures pour les différents signaux et on peut lire ainsi la valeur de temps :
   - **432.0 µsec** Temps d'exécution de la commande «Word_Write».

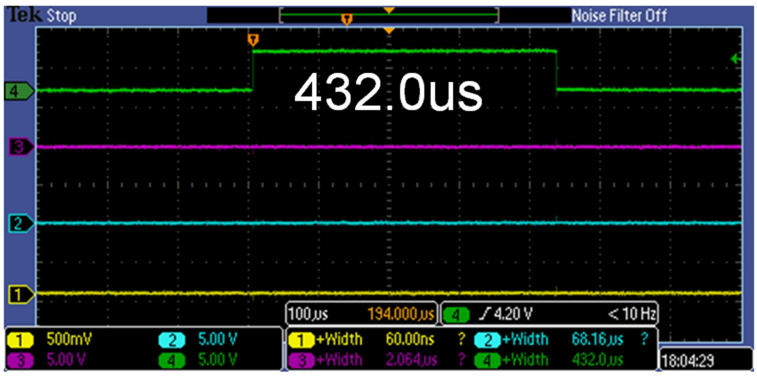


Figure : Capture d'écran de l'oscilloscope lors de l'écriture d'une valeur.

3.6 Test du cas d'utilisation «Lire des données» EXI_EDR_5 et 6

Objectifs du test

Le cas d'utilisation «Ecrire des donnes » se produit lors de la réception d'une trame CAN «Word_Read» et au niveau interne l'EDR décrite les valeurs reçus dans la trame pour réaliser une lecture sur la RAM pour obtenir une valeur donnée dans une adresse donnée.

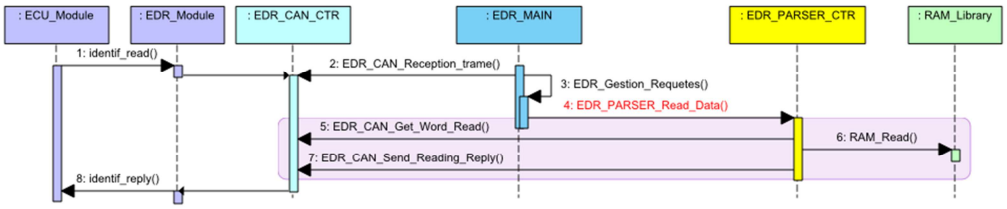


Figure : Diagramme de séquence du cas d'utilisation «Lire des données»

Moyens d'essais et configuration hardware et software

Pour la réalisation de ce test, on va utiliser la même **configuration hardware** décrite au début de ce chapitre.

Pour ce qui concerne la **configuration software** on se placera dans le fichier EDR_PARSER_CTR sur la fonction EDR_PARSER_Write_Data () pour ajouter une ligne d'activation/désactivation de sortie:

- Le pin **RB4** (Vert) pour mesurer le **temps d'exécution de la commande «Word_Read»**

```
void EDR_PARSER_Read_Data(unsigned long aIDtoSend){
IO_DEBUGGER_Set_LEDs(&LATB,0x10);
    // Unpack the CAN frame information to recover the read indicators.
    EDR_CAN_Get_Word_Read(&wordReadIndex);

    // Validate and read the value in the RAM at the addres received
    EDR_RAM_Read(&wordReadIndex, &wordRead);

    // Send the reading reply whit the index received (or 0xBADC0DE if error)
    // and the value stored in the RAM at this index
    EDR_CAN_Send_Reading_Reply(aIDtoSend, wordReadIndex, wordRead);
IO_DEBUGGER_Set_LEDs(&LATB,0x0);
}
```

Procédure utilisée

4. Réaliser la configuration matériel et logiciel comme décrit précédemment.
5. Ajouter les points de mesure avec les sondes de l'oscilloscope sur les pins configures au niveau logiciel :
   - La sonde 4 (vert) sur le pin RB4. (Mesure le temps d'exécution de la commande «Word_Read»).
6. Alimenter la carte EDR et observer la valeur des signaux. A l'aide de l'oscilloscope on peut ajouter des fonctions de mesures pour les différents signaux et on peut lire ainsi la valeur de temps :
   - **485.2 µsec** Temps d'exécution de la commande «Word_Read».



Figure : Capture d'écran de l'oscilloscope lors de la lecture d'une valeur.

3.7 Test du cas d'utilisation «Solliciter Registration» EXI_EDR_16 et 17

Objectifs du test

Le cas d'utilisation «Solliciter Information » se produit lors de la réception de une trame CAN «Recording_Reply». Le objectif initial de ce test et de **mesurer le temps** nécessaire pour réaliser la sauvegarde complète de tous les données de la RAM vers la EEPROM. Cette sauvegarde se réalise en plusieurs étapes et on désire mesurer aussi te temps de réalisation de chacune.

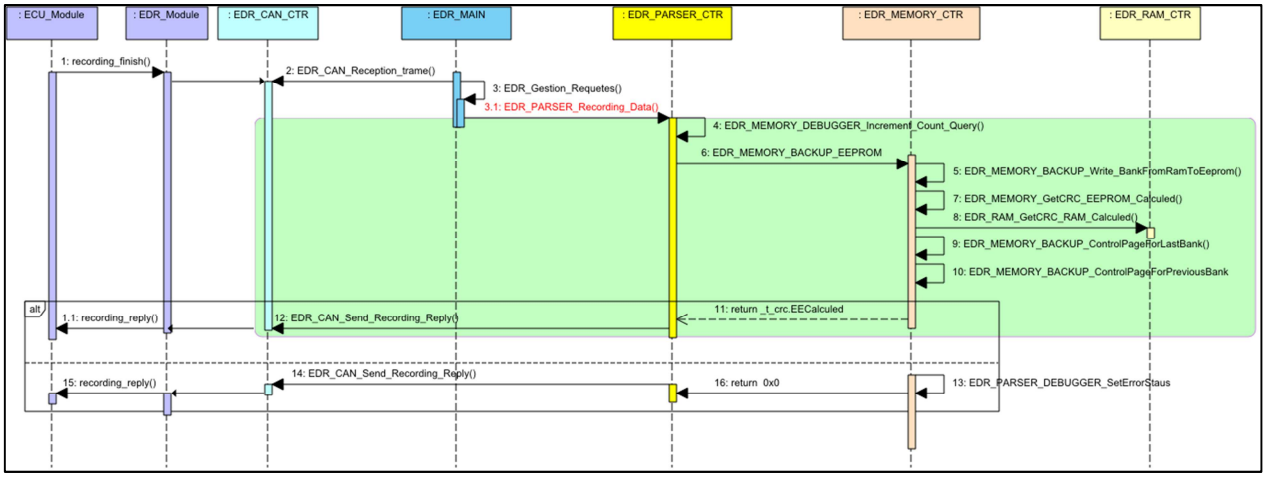


Figure : Diagramme de séquence du cas d'utilisation «Solliciter Registration»

Moyens d'essais et configuration hardware et software

Pour la réalisation de ce test, on va utiliser la même **configuration hardware** décrite au début de ce chapitre.

Pour ce qui concerne la **configuration software** on se placera dans le fichier EDR_MEMORY_CTR sur la fonction EDR_MEMORY_BACKUP_EEPROM () pour ajouter les lignes d'activation/désactivation des sorties:

- Le pin **RB4** (Jaune) pour mesurer **l'écriture des données RAM vers l'EEPROM**

```
IO_DEBUGGER_Set_LEDs(&LATB,0x10);
    EDR_MEMORY_BACKUP_Write_BankFromRamToEeprom();
IO_DEBUGGER_Set_LEDs(&LATB,0x0);
```

- Le pin **RB5** (Blue) pour mesurer la **relecture de l'EEPROM et calcul de son CRC**

```
IO_DEBUGGER_Set_LEDs(&LATB,0x20);
    _t_crc.EECalculed  = EDR_MEMORY_GetCRC_EEPROM_Calculed();
IO_DEBUGGER_Set_LEDs(&LATB,0x0);
```

- Le pin **RA0** (Violet) pour mesurer le **Calcul du CRC de la RAM**

```
IO_DEBUGGER_Set_LEDs(&LATB,0x20);
    _t_crc.RAMCalculed = EDR_RAM_GetCRC_RAM_Calculed();
IO_DEBUGGER_Set_LEDs(&LATB,0x0);
```

L'action final qui consiste à réaliser un test de cohérence prends un temps insignifiant no considérable.
Finalement pour mesurer le temps complet de tout le cas d'utilisation on se placera dans le fichier EDR_PARSER_CTR sur la fonction EDR_PARSER_Recording_Data () pour l'activation de une autre sortie

- Le pin **RA0** (Vert) pour mesurer le **temps total d'exécution de la commande «Recording_Finish»**

```
void EDR_PARSER_Recording_Data( unsigned long aIDtoSend){
IO_DEBUGGER_Set_LEDs(&LATA,0x1);
    // Increment the counter of Recording_Finish querys
    EDR_MEMORY_DEBUGGER_Increment_Count_Query();

    // Backup RAM to EEPROM and calcule, compare and provide CRC(0x0 if error).
    _t_crc.EECalculed = EDR_MEMORY_BACKUP_EEPROM();

    // Send the recording reply whit the CRC calculed from the EEPROM.
    EDR_CAN_Send_Recording_Reply(aIDtoSend, _t_crc.EECalculed);
IO_DEBUGGER_Set_LEDs(&LATA,0x0);
}
```

Procédure utilisée

1. Réaliser la configuration hardware décrite précédemment.
2. Inclure dans le code l'activation/désactivation des pins comme décrit précédemment.
3. Ajouter les points de mesure avec les sondes de l'oscilloscope sur les pins configures au niveau logiciel :
   - La sonde 1 (jaune) sur le pin **RB4**. (Mesure le temps de transfert de la RAM vers l'EEPROM)
   - La sonde 2 (bleu) sur le pin **RB5**. (Mesure le temps de calcul du CRC de l'EEPROM).
   - La sonde 3 (violet) sur le pin **RA0**. (Mesure le temps de calcul du CRC de la RAM).
   - La sonde 4 (vert) sur le pin **RA1**. (Mesure le temps total pour sauvegarder la mémoire).
4. Alimenter la carte EDR et observer la valeur des signaux. A l'aide de l'oscilloscope on peut ajouter des fonctions de mesures pour les différents signaux et on peut lire ainsi la valeur de temps :
   - **571.5 msec** Temps de transfert de la RAM vers l'EEPROM.
   - **436.8 msec** Temps de calcul du CRC de l'EEPROM.
   - **182.4 msec** Temps de calcul du CRC de la RAM.
   - **1.191 sec** Temps total pour sauvegarde de la mémoire.

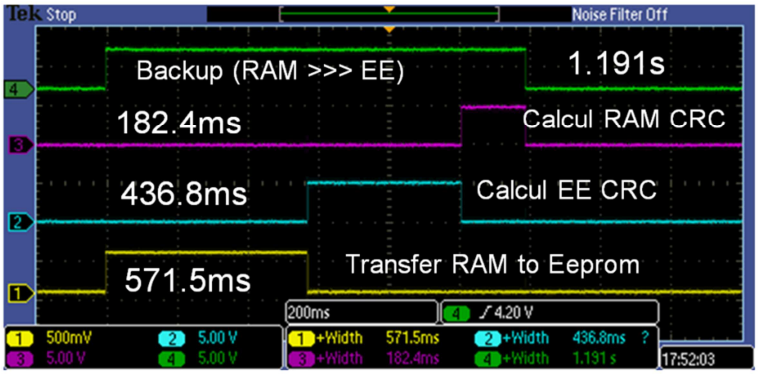


Figure: Capture d'écran de l'oscilloscope lors de la réalisation de la sauvegarde de mémoire.

Comme on avait indiqué, on observe que les 3 fonctions prennent la plus part du temps de l'exécution et la action finales (Test de Cohérence) prend un temps insignifiant en comparaison.

1191 msec – (571.5 msec + 436.8 msec + 182.5 msec) = 1191 msec – 1190.7msec = **0,3 msec**

On réalisera a continuation des tests pour vérifier que la sauvegarde des données et réalisé correctement, pour l'instant ce test nous a permis de connaitre le temps d'exécution pour vérifier que **notre application garanti les exigences** du cahier de charges de l'EDR au niveau de temps de réponse.

3.8 Test pour valider le changement de blocs de mémoire EEPROM

Objectifs du test

Une fois vérifie que les temps d'exécution des instructions restent inférieurs aux exigences attendues, il nous reste à valider que le mécanisme de changement de bloc mémoire après chaque initialisation fonctionne correctement et l'intégrité des données et maintenue tout au long du procès. Ce test va nous permettre de vérifier que:

- Les fonctions de calcul du CRC fonctionnent et retournent des valeurs connus attendus.
- Les compteurs s'incrémentent correctement après chaque enregistrement.
- Les données registres lors d'une session sont bien récupères après éteindre et rallumer l'EDR pour réaliser une nouvelle session. Ceci validera au même temps que
    - La séquence de sauvegarde s'est bien réalisée sur le banc le plus ancienne.
    - La restauration de la mémoire s'est bien réalisée depuis le banc récemment écrit.

Pour déterminer que les valeurs de CRC sont correctement calcules en utilisant le polynôme décrit dans le chapitre «Calcul des CRCs» on va s'appuyer sur les valeurs retournes pour une application tiers.

L'application **CRCTest.exe** (http://www.createwindow.com/programming/crc32/crcfile.htm) permet de calculer la valeur de un CRC sur 32bits (en utilisant le même polynôme que l'application EDR) à partir de un fichier texte rempli avec de caractères. Prenons par exemple 3 fichiers crées avec Notepad++, ils contiennent tous 2976 octets (Equivalent à 93 pages de 32 bytes de données de la RAM), seuls les 4 premiers changent:

- 2976_xFF_BYTES(CRC32_0x27A28498).txt
  FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF... jusqu'à 2976 BYTES .. le CRC est 0x27A28498
- DCBA_2972_xFF_BYTES(CRC32_0xEB5F4809).txt
  44434241FFFFFFFFFFFFFFFFFFFFFFFF... jusqu'à 2976 BYTES .. le CRC est 0xEB5F4809
- NullNullNullNull_2972_xFF_BYTES(CRC32_0x580CEEE8).txt
  00000000FFFFFFFFFFFFFFFFFFFFFFFF... jusqu'à 2976 BYTES .. le CRC est 0x580CEEE8

Moyens d'essais et configuration hardware et software

Pour la réalisation de ce test, on va utiliser la même **configuration hardware** décrite au début de ce chapitre.

La séquence d'actions que on va réaliser consiste en reproduire plusieurs sessions basiques:

- **Session 1**: Allumer la carte, On fait rien, Solliciter un Recording_Finish, Eteindre la carte.
- **Session 2**: Allumer la carte, Ecrire la valeur 0x41424344@0x0, Solliciter un Recording_Finish, Eteindre la carte
- **Session 3**: Allumer la carte, Ecrire la valeur 0x00000000@0x0, Solliciter un Recording_Finish, Eteindre la carte

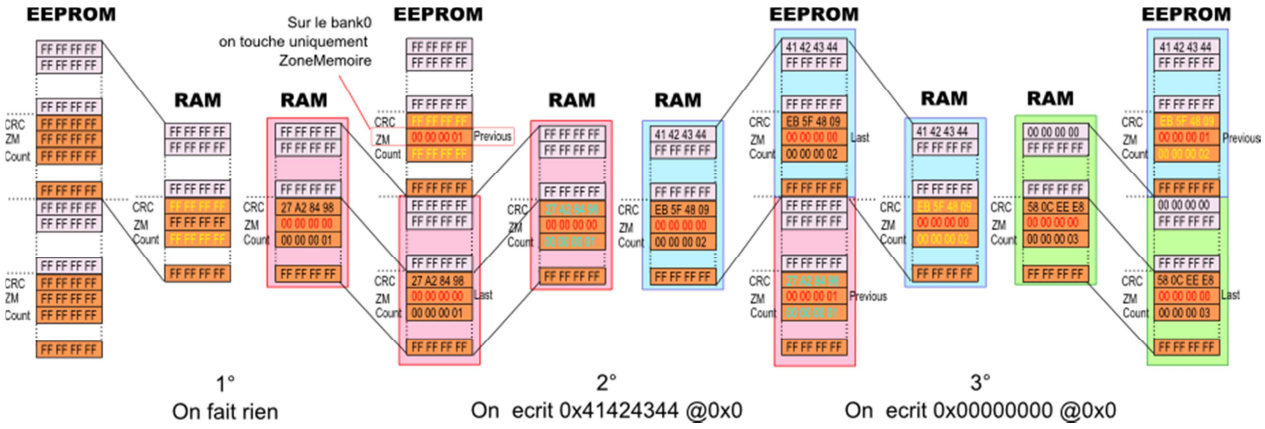


Figure: Séquence d'actions à réaliser et valeurs des CRCs et compteurs sur la Page de Contrôle.

Procédure utilisée

1. Réaliser la configuration hardware décrite précédemment
2. Configurer le logiciel PCAN-View selon indique dans le chapitre «Test pour valider la communication CAN 2.0A», ou bien récupérer la configuration à partir du fichier EDR_TEST_CAN_ADMIN.xmt.
3. Réaliser la séquence pour la première session.
    1. Alimenter la carte EDR.
    2. Initialiser l'application EDR à partir de MPLABx.
    3. Envoyer la trame ADMIN_QUERY (ID: 0x404) avec la commande pour **initialiser le banc 0 de la EEPROM**(Data:0xC1EABA0000000000). On doit recevoir une réponse avec la valeur «0x27A28498».
    4. Envoyer la trame ADMIN_QUERY (ID: 0x404) avec la commande pour **initialiser le banc 1 de la EEPROM**(Data:0xC1EABA0000000001). On doit recevoir une réponse avec la valeur «0x27A28498».
    5. Envoyer la trame **Recording_Finish** (ID: 0x403). (L'EDR doit réaliser le backup)
4. Réaliser la séquence pour la seconde session.
    1. Au lieu d'éteindre la carte EDR (qui nous obligerait a relancer l'application sur MPLABx) Envoyer la trame ADMIN_QUERY avec la commande qui sollicite la **restauration de la mémoire** EEPROM vers la RAM (0x00000004).
    2. Envoyer la trame ADMIN_QUERY avec la commande pour **solliciter le compteur de Recording_Finish** (0x00000004). On doit recevoir une réponse avec la valeur «1».
    3. Envoyer la trame ADMIN_QUERY avec la commande pour solliciter la **valeur du CRC registre sur la page de control du Banc 0 dans l'EEPROM** (Data:0x0000000300000000). On doit recevoir une réponse avec la valeur «0x27A28498».
    4. Envoyer la trame ADMIN_QUERY avec la commande pour solliciter la **valeur du CRC registre sur la page de control du Banc 1 dans l'EEPROM** (Data:0x0000000300000001). On doit recevoir une réponse avec la valeur «0x27A28498».
    5. Envoyer la trame **Word_Read** (ID: 0x402) pour lire la valeur sur l'indice 0 (Data:0x00000000). On doit recevoir une réponse avec la valeur «0xFFFFFFFF» (Valeur initial par défaut des données EEPROM).
    6. Envoyer la trame **Word_Write** (ID: 0x401) pour écrire la valeur 0x41424344 sur l'indice 0 (Data:0x0000000041424344).
    7. Envoyer la trame **Recording_Finish** (ID: 0x403). (L'EDR doit réaliser le backup)
5. Réaliser la séquence pour la troisième session.
    1. Envoyer la trame ADMIN_QUERY avec la commande qui sollicite la **restauration de la mémoire** EEPROM vers la RAM (0x00000004).
    2. Envoyer la trame **Word_Read** (ID: 0x402) pour lire la valeur sur l'indice 0 (Data:0x00000000). On doit recevoir une réponse avec la valeur «0x41424344» (Prouve l'intégrité des données).
    3. Envoyer la trame ADMIN_QUERY avec la commande pour **solliciter le compteur de Recording_Finish** (0x00000004). On doit recevoir une réponse avec la valeur «2». Valide l'incrément du compteur.
    4. Envoyer la trame ADMIN_QUERY avec la commande pour solliciter la **valeur du CRC registre sur la page de control du Banc 0 dans l'EEPROM** (Data:0x0000000300000000). On doit recevoir une réponse avec la valeur «0x27A28498».
    5. Envoyer la trame ADMIN_QUERY avec la commande pour solliciter la **valeur du CRC registre sur la page de control du Banc 1 dans l'EEPROM** (Data:0x0000000300000001). On doit recevoir une réponse avec la valeur «0xEB5F4809». Prouve que les derniers données on était registres sur ce banc et que le CRC a était calculé correctement.
    6. Envoyer la trame «**Identify_Read**» (ID: 0x400) pour récupérer l'information matériel, logiciel et la valeur du CRC des donnes récupérées de l'EEPROM. La valeur du CRC récupéré sera 0xEB5F4809.

3.9 Test pour valider le temps de mise en «mode opérationnel». EXI_EDR_1.

Objectifs du test

Selon la spécification **EXI_EDR_1** «*La transition entre le mode « Hors-tension » et « Opérationnel » ne doit pas dépasser 1s*». Cela implique qu'au niveau application l'EDR dispose de un seconde pour réaliser l'initialisation des modules de communication et restaurer les données mémoire de l'EEPROM vers la RAM avant de que l'ECU envoi à travers du bus CAN sa première requête. Ce test consiste justement à mesurer le temps que réellement est nécessaire pour réaliser ces actions et vérifier qu'on est en dessous du temps maxime requise dans l'spécification.
Dans un test réalise précèdent on avait mesure **le temps total pour restaurer la mémoire est 421.5 msec**

Moyens d'essais et configuration hardware et software

Pour la réalisation de ce test, on va utiliser la même **configuration hardware** décrite au début de ce chapitre sur la quel on ajoute une petit modification pour brancher une sonde de l'oscilloscope sur la source d'alimentation (qui vas nous permettre de déterminer le temps de la mise en alimentation) selon le schéma suivante.

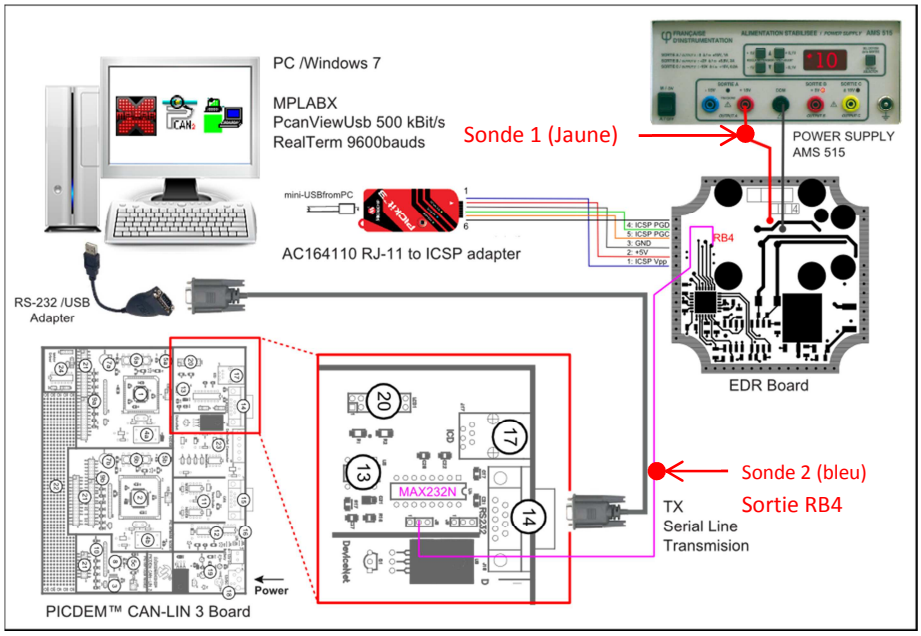


Figure: Connexion des sondes d'oscilloscope sur l'alimentation et la ligne série TX.

A niveau de **configuration software** on vas ajouter dans la fonction main() après la fonction d'initialisation EDR_MAIN_Initializations() et la fonction de restauration de la mémoire EDR_MEMORY_RESTORE_RAM() une fonction qui **envoie une chaine de caractères** a travers du port série.

Procédure utilisée

1. Réaliser la configuration hardware décrite précédemment en connectant les deux sondes d'oscilloscope.
2. Ajouter dans le main la fonction pour l'envoi d'une string par le port série et programmer le microcontrôleur
3. Eteindre puis rallumer la carte et observer les sorties à l'oscilloscope (escale de temps à 100ms).

```
void main(void)
{
  EDR_MAIN_Initializations();
  EDR_MEMORY_RESTORE_RAM();
  #ifdef TEST_LOG_SERIAL
  SW_UART_puts("\n\rInitialized!\n\r");
  #endif
  …
```

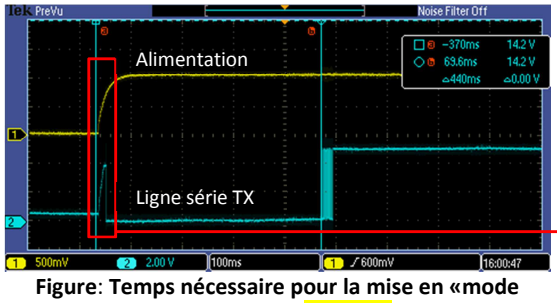


Figure: **Temps nécessaire pour la mise en «mode opérationnel»** (**440msec**).

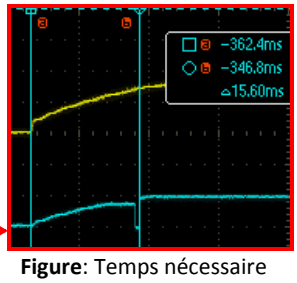


Figure: Temps nécessaire pour la configuration du module série (**15,6msec**).

3.10 Test pour mesurer le temps total de un cycle d'opération de l'EDR

Objectifs du test

Selon les spécifications de l'EDR une fois alimenté par l'ECU, l'EDR Syrena rentre en mode «Opérationnel», et doit fournir les services de communication, écriture, lecture et sauvegarde des données. Préalablement à niveau interne l'EDR a dû réaliser la restauration des données registres dans l'eeprom externe vers la RAM du microcontrôleur afin de répondre en vitesse aux requêtes d'écriture/lecture.

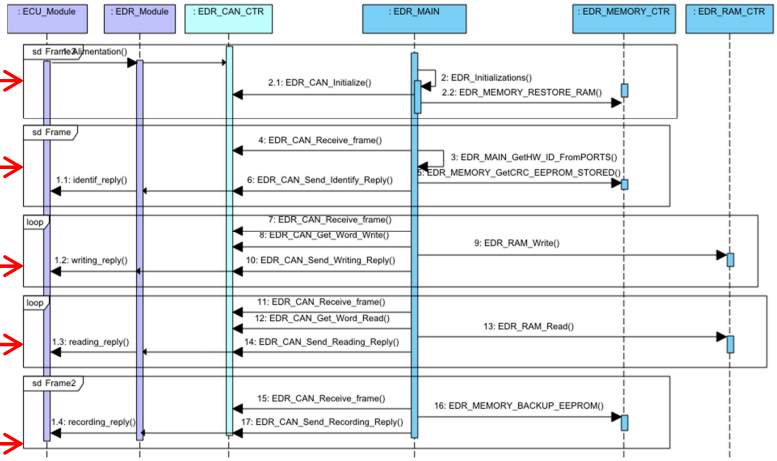
Avec ce test on désire mesurer le temps nécessaire pour réaliser un cycle complet de fonctionnement dans lequel on réalise une **écriture** sur chaque index de la RAM (Pour 93 pages de donnes, 93*8 = 744 index) et a continuation on réalise 774 opérations de **lecture** sur la totalité des index.

Moyens d'essais et configuration hardware et software

Pour la réalisation de ce test, on va utiliser la même **configuration hardware** décrite au début de ce chapitre.

Au niveau software on va créer une fonction de Test_EDR_PARSER_CTR_FullCycle_No_LOG() avec la séquence d'actions simulant une session de l'EDR depuis son initialisation jusqu'à la sauvegarde de donnes qui se déroule selon les étapes suivantes 5Pour mesurer le temps a l'oscilloscope on intercale l'activation/désactivation de une sortie RB4):

- **Restaurer la mémoire** de l'EEPROM vers la RAM.
- Exécuter les actions pour répondre à **1 requête** de «Identify_read»
- Exécuter une boucle d'écriture en répondant à **744 opérations** «word_write»
- Exécuter une boucle de lecture en répondant à **744 opérations** «word_read»
- Finaliser la sauvegarde des données et répondre à **1 requête** «recording_finish»



Chacune des opérations fera appel a la fonction EDR_PARSER_CTR_DEBUGGER_Test_Query qui a niveau interne indiquera au parser quel opération réaliser et avec quel paramètres. Cette opération a besoin de un temps pour préparer la simulation de la réception d'une trame CAN (au tour de 250 **µsec).**

Procédure utilisée

1. Ajouter dans le main ajouter la fonction `Test_EDR_PARSER_CTR_FullCycle_No_LOG();` pour lancer la séquence d'actions décrites précédemment.
2. Alimenter la carte EDR, lancer l'application sur MPLABx et observer la sortie à l'oscilloscope. Le temps total mesure est de **2,75 sec** (Les mesures à l'oscilloscope sont approximatives réalises a l'aide des curseurs).

	Opération	Estimation (msec)	Mesure Oscillo.	
1	Restaurer la mémoire	421.5 + 0,250 = 421,750	422	msec
2	1 requête de «Identify_read»	0,334 + 0,250 = 0,584	584	µsec
3	744 opérations «word_write »	744 * (432 + 0,250) = 507,400	536	msec
4	744 opérations «word_read »	744 * (485,2 + 0,250) = 546,900	584	msec
5	1 requête «recording_finish »	1191 + 0,250 = 1191.2	1,224	sec
	TOTAL		**2,750**	**sec**

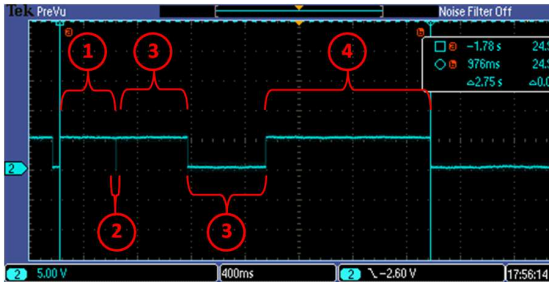


Figure: Temps total de un cycle d'opérations

3.11 Test pour valider le fonctionnement des compteurs.

Objectifs du test

On avait traité dans le chapitre «Fonctions ajoutes dans le Contrôleur de MEMOIRE» l'addition de deux compteurs d'évènements utilises dans les opérations de débogage:

- **Recording_Finish**: Le premier indique le numéro de requetés «Recording_finish» reçues.
- **Recording_OK**: Le second le numéro de sauvegardes de la RAM vers la mémoire EEPROM dans le bloc sélectionne qui se sont réalisés avec succès.

La différence entre ces deux compteurs permet de déduire le nombre de sauvegardes qui s'ont produit avec erreur et peut être une indication de que

- la mémoire EEPROM commence à présenter des **défaillances d'écriture/lecture**.
- il s'a produit une **coupure d'alimentation**, entre le moment de recevoir la requête de «Recording_finish» et la fin de la sauvegarde après la quel on incrément et on garde le compteur dans la page de control de l'EEPROM

Moyens d'essais et configuration hardware et software

Pour tester que les compteurs registrent les évènements tel comme on l'attend, il serait nécessaire dans le premier cas de rentrer en mode débogage et attendre une défaillance de l'EEPROM et cela peut entrainer un temps considérable. Par contre, on peut produire une coupure d'alimentation de la carte juste au moment après avoir envoyé une requête et avant de finaliser la sauvegarde. Couper de façon manuel l'alimentation juste après l'envoi de la trame résulte assez délicat puisque le temps disponible est très court.

Pour cela on ajoutera un «**breackpoints**» dans le code juste après avoir reçu la requête «Recording_finish» et avoir incrémenté le compteur correspondant, une fois l'application arrive à ce point on coupera l'alimentation.

```
void EDR_PARSER_Recording_Data( unsigned long aIDtoSend)
{
    // Increment the counter of Recording_Finish querys
    EDR_MEMORY_DEBUGGER_Increment_Count_Query();

    // Backup RAM to EEPROM and calcule, compare and provide CRC(0x0 if error).
    _t_crc.EECalculed = EDR_MEMORY_BACKUP_EEPROM();

    // Send the recording reply whit the CRC calculed from the EEPROM.
    EDR_CAN_Send_Recording_Reply(aIDtoSend, _t_crc.EECalculed);
}
```

A niveau de la **configuration hardware** il suffira de réaliser le schéma suivait déjà présente en test précédents.

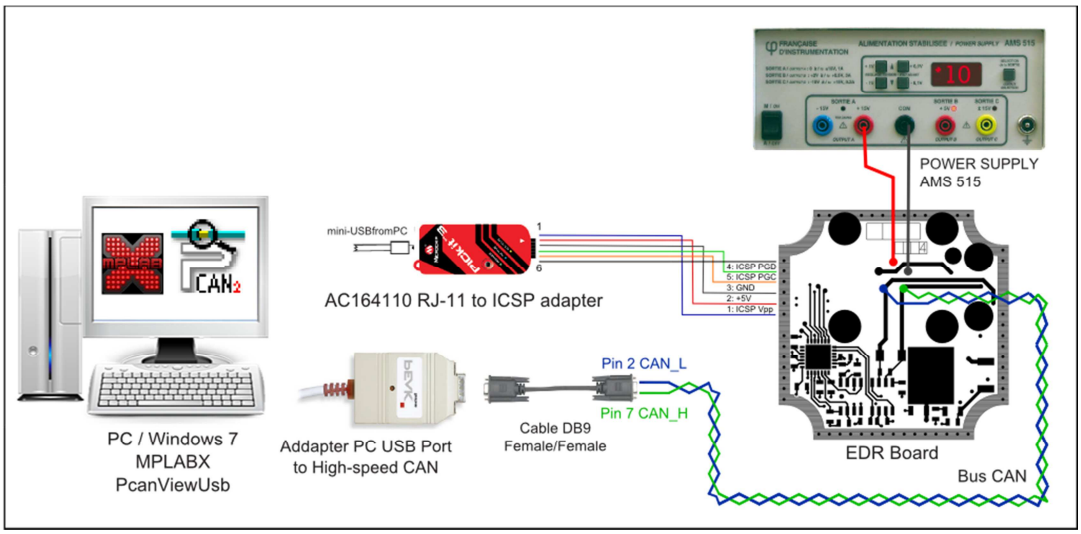


Figure: Connexions de la carte EDR avec le PC et l'alimentation stabilisée.

Procédure utilisée

1. Dans le fichier EDR EDR_PARSER_CTR.c on ajoute un «breackpoint» dans la ligne indique précédemment.
2. Alimenter la carte EDR. **Première session du test.**
    1. Initialiser l'application EDR à partir de MPLABx.
    2. Avec l'application PCAN-View, envoyer la trame ADMIN_QUERY avec la commande pour solliciter le **compteur de Recording_Finish (**0x00000004). On doit recevoir une réponse avec la valeur «X».
    3. Envoyer la trame ADMIN_QUERY avec la commande pour solliciter le **compteur de Recording_OK** (0x00000005). En principe, on doit recevoir une réponse avec la même valeur «X».
    4. Envoyer la trame **Recording_Finish** (ID: 0x403).qui fera arrêter l'application (`Halting... Target Halted`) dans le point d'exécution que on avait indiqué avec le «breackpoint»
    5. Couper l'alimentation de la carte EDR. Cela **implique que l'application est arrêtée sans avoir réalisé la sauvegarde de mémoire**.
3. Alimenter la carte EDR. **Seconde session du test.**
    1. Initialiser l'application EDR à partir de MPLABx.
    2. Avec l'application PCAN-View, envoyer la trame ADMIN_QUERY avec la commande pour solliciter le **compteur de Recording_Finish** (0x00000004). On doit recevoir une réponse avec la valeur «**X+1**».
    3. Envoyer la trame ADMIN_QUERY avec la commande pour solliciter le **compteur de Recording_OK** (0x00000005). On doit recevoir une réponse avec la même valeur qu'avant «**X**» impliquant que le compteur n'a pas été incrémenté et la sauvegarde de donnes n'est pas valide (interrompu dans notre cas).

3.12 Test pour valider le temps de vie d'une case mémoire.

Objectifs du test

Dans la carte EDR on utilise une mémoire externe EEPROM de 8ko résistent à hautes températures modèle 25LC640A du fabricant Microchip. Selon les spécifications de cette mémoire elle doit supporter **1.000.000 de cycles d'écriture** à température ambiante. Dans le test suivant, on va procéder à réaliser une vérification partial selon le principe suivant.

- Cette mémoire utilise une organisation de 8 bits (1byte) et dans notre application l'EDR travaille avec valeurs de 32bits adressables par indices vers le premier byte des 4 qu'héberge une variable. Les pages mémoire sont de 32 bytes, donc celle-ci tiens une organisation de 8192/32 = **256 Pages**.
- Dans l'EDR on divise en deux l'eeprom pour obtenir 2 blocs de mémoire.
- Un bloc mémoire de l'eprom et limite par la taille disponible pour créer une table de la même taille sur la RAM. La RAM disponible dans le PIC1826K80 est de 3649 (0xE41) bytes
- Dans la configuration actuelle l'application gère **94 pages**, et le total des variables application occupe le 88% de la RAM et restent disponibles 431 (0x1AF) bytes. Cela nous permet encore d'augmenter encore la mémoire en 431/32= 13,375 Pages.
- Prenons une marge de sécurité pour considérer le cas ou en utilisant le maximum d'espace on pourra créer une table de 94+14= 108 pages. Cela nous donnera un table de 108x32=3456 bytes. Et pour pouvoir la registrer dans l'eeprom en deux banc différents on aurait besoin de un espace de 2x108 = **216 pages**.
- Donc in nous reste disponibles les dernières **20 pages** que on n'utilisera pas dans l'application EDR sur lesquelles on pourra réaliser le test pour valider le temps de vie d'une cellule mémoire.

Feature	Value
Max. Write Current	5 mA at 5.5V, 10 MHz
Read Current:	5 mA at 5.5V, 10 MHz
Standby Current:	1 mA at 5.5V
Organization	8192 x 8-bit
Page	32 Byte
Self-Timed Erase and Write Cycles	5 ms max
Endurance:	1,000,000 erase/write cycles
Data retention:	> 200 years
ESD protection:	> 4000V

Table: Caractéristiques de la mémoire : Microchip 25LC640A 64K SPI Bus Serial EEPROM

Moyens d'essais et configuration hardware et software

Pour la réalisation de ce test, on va utiliser la même **configuration hardware** décrite au début de ce chapitre.
Pour la **configuration software**, (selon le principe qu'on garde des variables de 4 bytes) on va choisir initialement les 4 premiers bytes de la dernier page de l'eeprom situées à l'adresse 8192 - 32 = **8160**. (L'avant dernière a 2036, …)

Dans le fichier EDR_TEST on ajoutera la fonction **Test_GRILL_4Bytes()** qui réalisera en boucle pendant 1.500.000 de fois les opérations suivantes:

1. **Activer une sortie** pour mesurer le temps d'écriture/lecture à l'oscilloscope.
    1. Ecriture d'une variable de 4 bytes sur l'EEPROM (avec une fonction bas niveau.)
    2. Lecture de la valeur écrite précédemment dans l'EEPROM
    3. Test de cohérence pour valider l'écriture/lecture
2. **Désactiver la sortie** pour mesurer le temps de transmission CAN de la trame de réponse
    1. Envoi d'une trame CAN avec l'ID 0x304, le numéro de itération et le résultat du test (1:Ok, 0: Erreur).

Procédure utilisée

1. Réaliser la configuration hardware décrite précédemment
2. Dans le fichier EDR_Test.h on ajoute un define pour activer les fonctions de Test qui vont permettre de envoyer des lignes de log a travers du port série.
   ```
   #define TEST_LOG_SERIAL
   ```
3. Dans le fichier EDR_Test.c on vérifie que la fonction est bien appelle dans le test conditionnel du define:
   ```
   #ifdef TEST_LOG_SERIAL
       Test_GRILL_4Bytes();
   #endif
   ```

Premier test à **1000 écritures @ 8160** pour **estimer le temps de duration** de 1.000.000 d'écritures.

4. Dans la fonction Test_GRILL_4Bytes() fixer la fin boucle a 1000 itérations.
5. Configurer le logiciel PCAN-View et redémarrer le mode Trace qui va permettre de registrer toute les trames qui seront reçues à continuation (Menu Trace/Start).
6. Alimenter la carte EDR, redémarrer l'application et observer :
   1. Les trames reçus dans l'onglet Trace de l'application PCAN-View. La premier itération était réalisé dans le temps 866,2519sec, l'itération 1000 dans le temps 870,5960sec après 4,3083 secondes. Selon calcules une écriture aurait durée 4,309 millisecondes, en 1.000.000 d'écritures prendre **71.81 minutes.**
   2. La signal de sortie à l'oscilloscope. On observe qu'un cycle prend 4,33 millisecondes (mesure avec les curseurs).

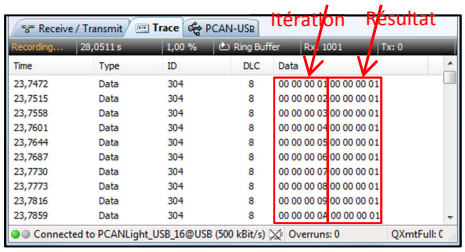


Figure: Registre de trames reçues

Temps	Unité	Ecritures
866.2519	seconde	0
870.5602	seconde	1.000
4.3083	secondes	1.000
0.0043083	secondes	1
4309	secondes	1.000.000
71.81666	minutes	1.000.000

Table: Temps observe et stimmées.

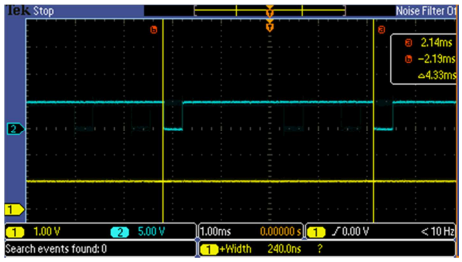


Figure: Capture a l'oscilloscope

Seconde test à **1.500.0000 écritures/lectures @ 8160**

1. Dans la fonction Test_GRILL_4Bytes() fixer la fin boucle a 1.500.0000 itérations.
2. Configurer le logiciel PCAN-View et redémarrer le mode Trace qui va permettre de registrer toute les trames qui seront reçues à continuation (Menu Trace/Start).
3. Alimenter la carte EDR, redémarrer l'application et attendre la fin des itérations après le temps stimmé (4.3083*1500000 / 60 = 107,7075 minutes). Ce tes nous a permis de valider que après 1.500.000 (0x16E360) itérations le résultat d'une écriture/lecture pour une variable de 4 bytes était toujours positive.
   (La consommation de la carte était stabilise au tour de 0.0016 A.)

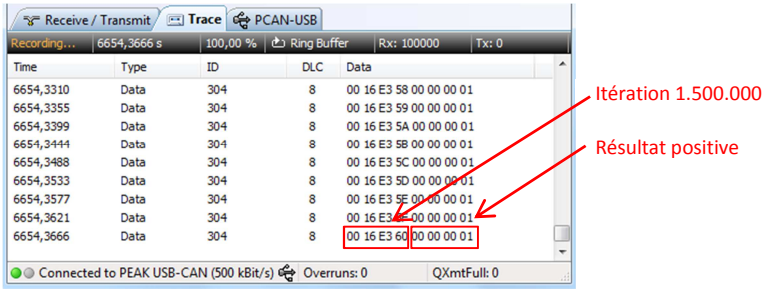


Figure: Registre des dernières trames reçues après 1.500.000 itérations.

3émé test à **5.000.0000 écritures/lectures @ 8160**.
Même procédure que dans le test précède. Résultat toujours positive, pas d'erreurs.

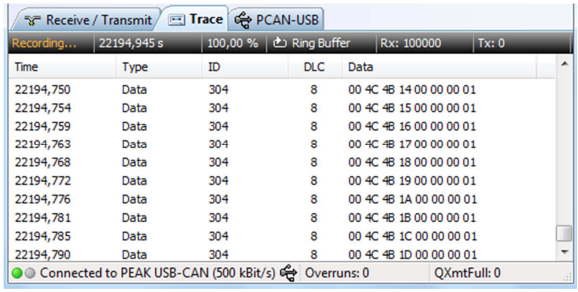


Figure: Registre des dernières trames reçues après 5.000.000 itérations.

4éme Test **1.500.000 effacement/écritures/lectures @ 8191**
Dans ce nouveau test on va ajouter dans l'itération une étape pour effacer la variable.

1. Effacement d'une variable de 4 bytes (une fonction bas niveau écrit la valeur 0xFFFFFFFF.)
2. Ecriture d'une variable de 4 bytes (avec la valeur du compteur du nombre d'itérations.)
3. Relecture de la variable
4. Comparution avec le test de cohérence pour valider l'écriture/lecture

Puisque il existe une procédure différent, on réalisera ce tes dans une adresse mémoire différent (@8191, le dernier byte de la mémoire EEPROM).Résultat toujours positive, pas d'erreurs.

5éme Test **2.000.000 effacement/écritures/lectures @ 8191**
Même procédure que dans le test précède. Résultat toujours positive, pas d'erreurs.

6émé Test **4.500.000 effacement/écritures/lectures @ 8191**
Même procédure que dans le test précède. Finalement des résultats négatifs sont apparus.
Le logiciel PCAN-View ne pouvant registré que dans un fichier de log les derniers 100000 trames on ne pourra pas déterminer à partir de quel moment ils se sont produit les erreurs.

```
 99976)  33643925.7  Rx         0304  8  00 3D 08 E8 00 00 00 01
 99977)  33643934.1  Rx         0304  8  00 3D 08 E9 00 00 00 01
 99978)  33643942.4  Rx         0304  8  00 3D 08 EA 00 00 00 01
 99979)  33643950.8  Rx         0304  8  00 3D 08 EB 00 00 00 01
 99980)  33643959.2  Rx         0304  8  00 3D 08 EC 00 00 00 01
 99981)  33643967.6  Rx         0304  8  00 3D 08 ED 00 00 00 01
 99982)  33643976.0  Rx         0304  8  00 3D 08 EE 00 00 00 01
 99983)  33643984.3  Rx         0304  8  00 3D 08 EF 00 00 00 01
 99984)  33643992.7  Rx         0304  8  00 3D 08 F0 00 00 00 00
 99985)  33644001.1  Rx         0304  8  00 3D 08 F1 00 00 00 00
 99986)  33644009.5  Rx         0304  8  00 3D 08 F2 00 00 00 00
 99987)  33644017.9  Rx         0304  8  00 3D 08 F3 00 00 00 00
 99988)  33644026.2  Rx         0304  8  00 3D 08 F4 00 00 00 00
 99989)  33644034.6  Rx         0304  8  00 3D 08 F5 00 00 00 00
 99990)  33644043.0  Rx         0304  8  00 3D 08 F6 00 00 00 00
 99991)  33644051.4  Rx         0304  8  00 3D 08 F7 00 00 00 00
 99992)  33644059.8  Rx         0304  8  00 3D 08 F8 00 00 00 01
 99993)  33644068.1  Rx         0304  8  00 3D 08 F9 00 00 00 01
 99994)  33644076.5  Rx         0304  8  00 3D 08 FA 00 00 00 01
 99995)  33644084.9  Rx         0304  8  00 3D 08 FB 00 00 00 01
 99996)  33644093.3  Rx         0304  8  00 3D 08 FC 00 00 00 01
 99997)  33644101.6  Rx         0304  8  00 3D 08 FD 00 00 00 01
 99998)  33644110.0  Rx         0304  8  00 3D 08 FE 00 00 00 01
 99999)  33644118.4  Rx         0304  8  00 3D 08 FF 00 00 00 01
100000)  33644126.8  Rx         0304  8  00 3D 09 00 00 00 00 00    <- Iteration 4.000.000
```

On observe qu'il y a séquences de 8 erreurs consécutives suivis de 8 itérations correctes, ce comme si dans le BYTE qu'on prétende effacer il y aurait le BIT 4 sur lequel la mémoire n'est plus capable de le placer en état 0.

3.13 Test avec une application permettant de simuler la communication avec l'ECU.

Objectifs du test

Apres avoir vérifié le comportement de un cycle de vie de l'application EDR à partir d'une fonction interne, comme on a décrit dans un test précèdent, ce test va permettre de valider qu'une autre **application externe** s'exécutant dans une autre carte, en communication a travers du bus CAN, peut dialoguer avec l'EDR selon les spécifications requises. Cette application externe sera un sorte de **simulateur de l'application de l'ECU** qui va solliciter une série de services a l'EDR a travers d'un ensemble de requetés CAN.

Avec ce test on désire **mesurer le temps** nécessaire pour **réaliser un cycle complet** de fonctionnement dans lequel l'application simulant l'ECU :

- Réalise une requête de «Identify_read»
- Réalise une **écriture** sur chaque index de la RAM (Pour 93 pages de donnes, 93*8 = 744 index) et a continuation
- Réalise 774 opérations de **lecture** sur la totalité des index.
- Finalise avec l'envoi d'une requête «Recording_finish»

Moyens d'essais et configuration hardware et software

Pour la **configuration hardware** on fera tourner l'application simulant l'ECU dans la carte de développement **PICDEM CAN-LIN 3** qu'on connectera a un bus CAN sur lequel se trouveront aussi connectes la **carte EDR** et **l'analyseur de bus CAN** qui nous permettra d'espionner toute les trames que circuleront autan de l'EDR que du simulateur ECU.

Pour la **configuration software** on utilisera l'application EDR tel quel a la quel on ajoutera un **#define ECU** pour venir modifier le code uniquement dans la fonction **main()** de façon à ajouter la séquence de requetés comme on a décrit.

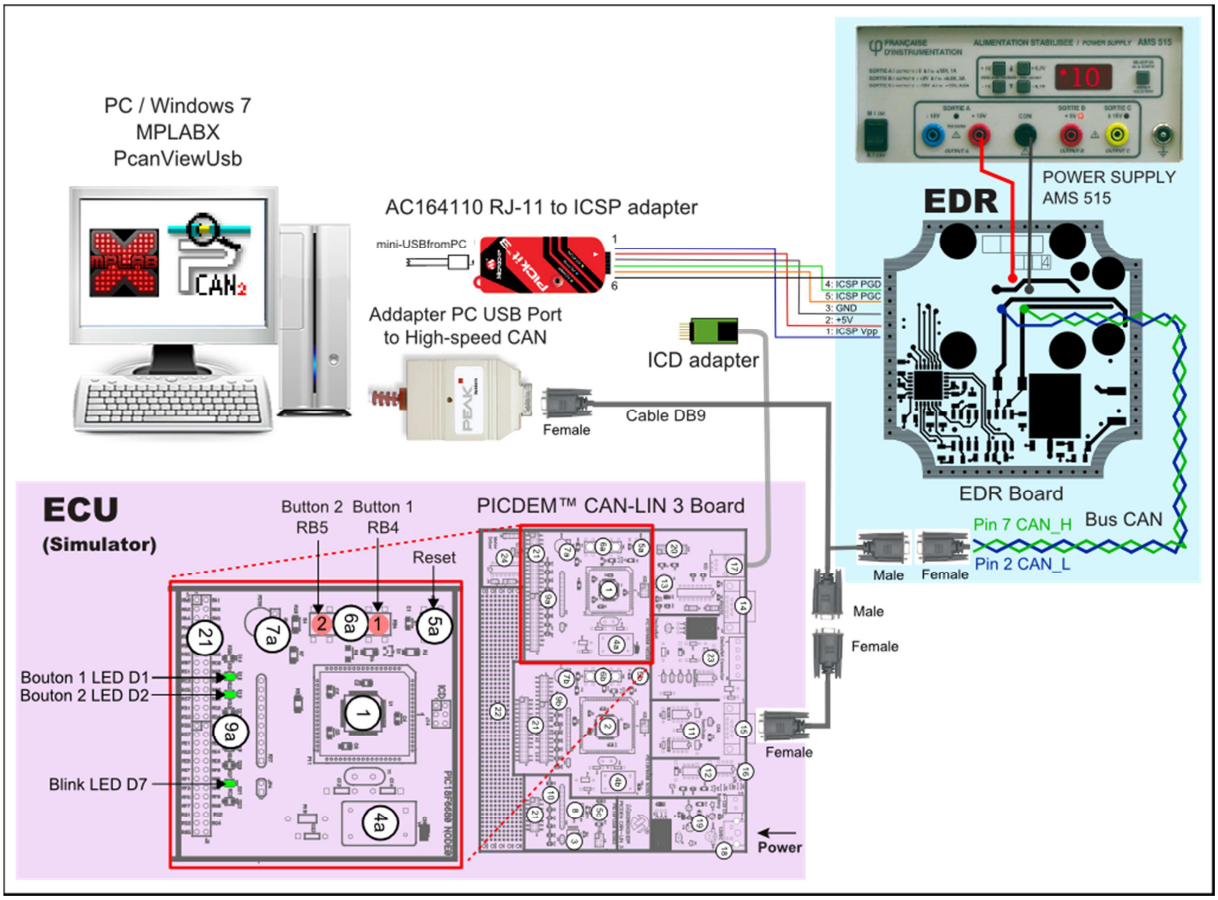


Figure: Connexion de la carte EDR avec la carte PICDEM CAN-LIN 3.

Pour les essais de mise en œuvre du bus CAN, les moyens nécessaires sont :

- Une carte EDR (Programme avec l'application EDR sur le microcontrôleur)
- Une carte d'évaluation PICDEM CAN-LIN3 (sur la quel tournera le simulateur de l'ECU).
- Le programmateur debugger PICKit3
- L'adaptateur ICSP AC164110 **6-pins femelles /RJ-11** utilise pour la programmation de la carte PICDEM.
- Le logiciel MPLABX avec le compilateur xC8.
- L'adaptateur "**PC USB Port to High-speed CAN**" de **PEAK**-System Technik GmbH (www.peak-system.com)
- Le logiciel fourni avec l'adaptateur CAN : PcanViewUsb.exe (Version 3.2.3.221)
- Une alimentation stabilisée a 10V pour alimenter l'EDR
- Un câble spécial ou adaptateur multiple pour connecter les différents connecteurs DB9 pour le port CAN de chaque dispositif.

Comme interface avec l'utilisateur, la carte PICDEM CAN-LIN 3 dispose de un **array de LEDs** connecte avec le PORTD et **2 boutons** connectés aux pins RB4 et RB5. On utilisera ces entres/sorties pour établir le comportement de l'application simulant l'ECU :

- L'appui du **bouton 1**, allume le **LED 1** et lance le cycle de requetés destines a l'EDR (1 Identification, 744 écritures, 744 lectures et 1 «Recording finish»)
- L'appui du **bouton 2**, allume le **LED 2** et envoie une trame de debug pour solliciter à l'EDR le nombre de «Recording finish» réalisés avec succès.

  EDR_CAN_DEBUGGER_Send_Admin_Reply(iD_Q5_DEBUGGER_ADMIN_QUERY,GET_COUNT_REC_OK,EMPTY);
- Le **LED 7** clignotera en permanence indiçant que l'application est en fonctionnement.

Avant d'aborder la séquence simulant l'ECU dans la fonction main() il nous faudra ajouter un petit détail dans la fonction **EDR_PARSER_CONTROLLER_Initialize** du fichier EDR_PARSER_CTR.c qui consiste à initialiser le contrôleur CAN de mode a accepter les trames avec les identificateurs de réponse de la part de l'EDR.

```
#ifdef EDR
   EDR_CAN_CONTROLLER_Initialize(iD_Q1, iD_Q2, iD_Q3, iD_Q4, iD_QA_DEBUGGER);
#endif
#ifdef ECU
   EDR_CAN_CONTROLLER_Initialize(iD_R1, iD_R2, iD_R3, iD_R4, iD_RA_DEBUGGER);
#endif
```

Dans la fonction **main()** a l'intérieur d'un **#define ECU** on ajoutera les variables et fonctions suivantes pour implémenter les fonctions basiques pour simuler l'ECU. Pour ce qui concerne la transmission de trames de la part de l'ECU on utilisera la fonction déjà écrite EDR_CAN_DEBUGGER_Send_Admin_Reply qui reçoit comme paramètres l'ID et les valeurs selon la requête qu'on désire réaliser.

Variable	Taille	Type	Observation
ecu_button1WasPressed	8 bits	unsigned char	Indique que le bouton 1 a été appuie.
ecu_button2WasPressed	8 bits	unsigned char	Indique que le bouton 2 a été appuie.
ecu_ACK	8 bits	unsigned char	Utilise pour simuler une réception de trame
ecu_heartbeatCount	8 bits	unsigned char	Conteur utilise pour le battement du LED de fonctionnement.

Fonction	Observation
unsigned char ECU_Button1Pressed(void);	Retourne TRUE si le bouton 1 est presse pour lancer la séquence de requêtes.
unsigned char ECU_Button2Pressed(void);	Retourne TRUE si le bouton 2 est presse pour lancer la commande GET_COUNT_REC_OK
void ECU_SimulateReception(void);	Attend la réception d'une trame CAN et vide les buffers de réception RX
void ECU_Heartbeat(void);	Décrémente un conteur qui détermine le changement d'état du LED 7.

La fonction **ECU_SimulateReception** est nécessaire afin d'éviter l'envoi massive de requêtes de la part de l'ECU sans avoir attendu une réponse de l'EDR. Cela donnera un échange de trames ordonne que on pourra analyser plus tard.

Procédure utilisée

Programmation préalable des applications dans chaque carte

- Programmation de **l'application EDR** dans la carte EDR
  - Dans le fichier vérifier la présence de la directive #define EDR
  - Connecter le câble ICD sur l'EDR, Alimenter la carte EDR puis Cliquer sur le bouton Make and Program Device Main Project
  - Eteindre la carte EDR, Déconnecter le câble ICD et rallumer la carte.
- Programmation de **l'application ECU** dans la carte PICDEM CAN-LIN 3
  - Dans le fichier vérifier la présence de la directive #define ECU
  - Connecter le câble ICD sur la carte PICDEM, Alimenter la carte PICDEM puis Cliquer sur le bouton Make and Program Device Main Project
  - Eteindre la carte PICDEM, Déconnecter le câble ICD et rallumer la carte.

Lancer **l'application pour analyser le bus CAN**

- Sur le PC, lancer l'application PCAN-View
- Sur l'application PCAN-View cliquer sur l'icône pour réaliser un reset et effacer dans l'onglet «Receive/Transmit» toute les trames présents de test précédents.
- Sur l'onglet «Trace», si une opération de log a commencé, appuyer sur l'icône stop pour l'arrêter.
- Appuyer sur l'icône start pour effacer les logs précédents et commencer l'enregistrement de un nouveau.

Lancer une **séquence de requêtes** de la part du simulateur **ECU**

- Appuyer sur le bouton 1 de la carte PICDEM CAN-LIN 3
- Observer l'échange de trames entre les deux cartes dans l'onglet «Receive/Transmit» de l'analyseur CAN PCAN-View. Avec temps indiqué dans l'onglet «Trace» on déduit une duration totale de **3,1925 secondes**.

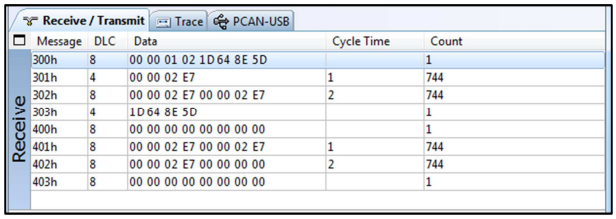


Figure: Total des trames registres par l'analyseur du bus CAN, envoyés par l'ECU (40x) et envoyés par l'EDR (30x)

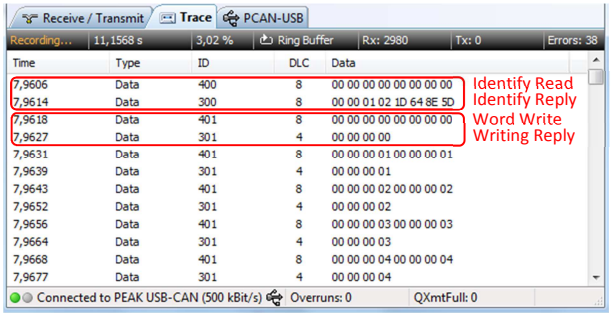


Figure: Séquence temporel des trames registres par l'analyseur du bus CAN

Lancer une **pétition pour connaitre le nombre d'enregistrements valides** de la part du simulateur **ECU**

- Appuyer sur le bouton 2 de la carte PICDEM CAN-LIN 3
- Observer dans l'onglet «Receive/Transmit» l'apparition de 2 nouveaux trames. La pétition envoyé par l'ECU avec l'ID ox404 et la réponse envoyé par l'EDR avec l'ID 0x304.

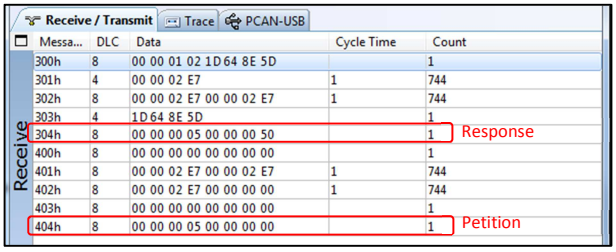


Figure: Apparition des trames avec la pétition du nombre d'enregistrements (envoyé par l'ECU) et la réponse (envoyé par l'EDR).

	Opération	Diff.Temps	
2	1 requête de «Identify_read»	0,0008	sec
3	744 opérations «word_write »	0,9297	sec
4	744 opérations «word_read »	1,0540	sec
5	1 requête «recording_finish »	1,2098	sec
	TOTAL	3,1925	sec

Table: Temps partiaux et total d'exécution. (Les mesures tiennent en compte le moment d'apparition des trames dans le bus CAN et calcule la différence entre la première requête et la dernier réponse)

4 Réalisation de tests environnementaux.

4.1 Test température

Objectifs du test

On désiré étudier le comportement de la carte EDR et mesurer la température de fonctionnement des composants lors de l'exécution de l'application dans différents conditions ambiance. Le principe consiste à utiliser des sondes de température (modèle PT100) colles aux différents composants et valider que l'application tourne à l'intérieur de un four à température régulée et les dispositifs respectent la spécification **d'exécution a hautes températures**.

Pédant le test on fait exécuter en boucle la fonction de test décrite dans le chapitre «Test pour mesurer le temps total de un cycle d'opération de l'EDR » qui envoie le long de l'exécution de trames CAN de réponse avec l'identificateur et les paramètres de retour pour l'ECU.

Les paramètres qu'ont désiré mesurer lors d'une session de test sont:

- **L'état de l'application**: Pour cela on pourra observer
  - l'état de la sortie B4 qu'on fera varier dans chaque étape d'exécution.
  - l'ID reçu dans les trames de réponse envoyée par l'EDR
- L'**alimentation** de la carte EDR pendant l'exécution du test.
- La **consommation en courant** lors des différents états.
- La **température externe a l'EDR** que correspond avec la température à l'intérieur du four.
- La **température** des différents composants:
  - Température du **MICROCONTROLEUR** `(U1 MICRO 64KBITS 28PTS B:QFN-S -40°/150° PIC18F26K80)`
  - Température de la mémoire **EEPROM** `(U2 MEMOIRE EEPROM SPI 64K SOIC8 -40/150° REF:25LC640A)`
  - Température du **Transceiver CAN** `(U3 RECEPTEUR CAN 1MBd SO8 -40/150° REF:TLE6250)`
  - Température du **Régulateur** `(REGULATEUR AJUSTABLE 1.2>37V 1.5A D2PAK -55/+150°)`

Etat	ID reçu	Alimentation	Courant	T°C	U1 T°C	U2 T°C	U3 T°C	U4 T°C
Restauration	0x300	+10V		25°C				
Writing	0x301	+10V		25°C				
Reading	0x302	+10V		25°C				
Backup	0x303	+10V		25°C				

Table: Valeurs moyens de courant et température à mesurer à chaque état de l'application.

Moyens d'essais et configuration hardware et software (CONFIGURATION 1)

L'idée est de récupérer les différents logs de l'application d'analyse pour réaliser une graphique unique.

Pour la réalisation du test il nous sera nécessaire introduire dans un four a température contrôle la carte EDR sur la quel se trouvent les sondes de température collés aux composants que on désire analyser. Le câblage doit également supporter des hautes températures et sortir à travers d'un orifice vers les dispositifs de alimentation et mesure. Comme matériel nécessaire pour réaliser le test on utilisera:

- Une **source d'alimentation stabilisée** a voltage variable entre +8 et +12v.
- Un **oscilloscope** pour la lecture de voltage, courent et éventuellement l'statut marque par une sortie.
- Un **analyseur de bus CAN** (pour recevoir les trames indican le fonctionnement et statuts de l'EDR)
- Una **unité de mesure USB TC-08** Thermocouple Data Logger du fabricant Pico Technology
  http://www.picotech.com/thermocouple.html
- De **sondes de température** de type XF-321-FAR - CAPTEUR T/COUPLE K PTFE 1M MINIPLUG du fabricant LABFACILITY et distribue par Farnell.
  http://fr.farnell.com/labfacility/xf-321-far/capteur-t-couple-k-ptfe-1m-miniplug/dp/4100748
- Un PC de control pour récupérer de l'information pendant le test avec les logiciels :
  - **PCAN-View** qui permettra d'obtenir un log avec le temps de réception de chaque trame.
  - **PicoLog** qui registrera également dans un log les températures des composants mesures a un intervalle de temps que l'on pourra fixer.

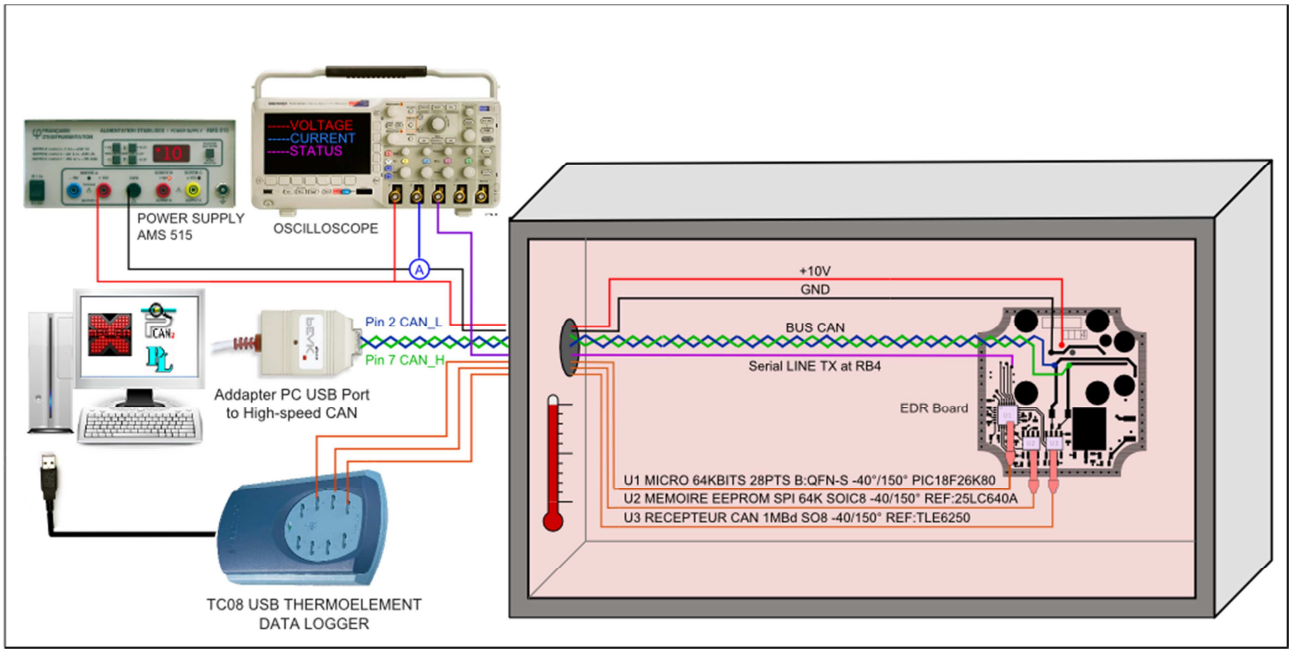


Figure: Connexions entre la carte EDR et les différents éléments de log et mesure.

Le câblage est décrit dans l'schéma précèdent …

(Reste à définir la mesure du courant)

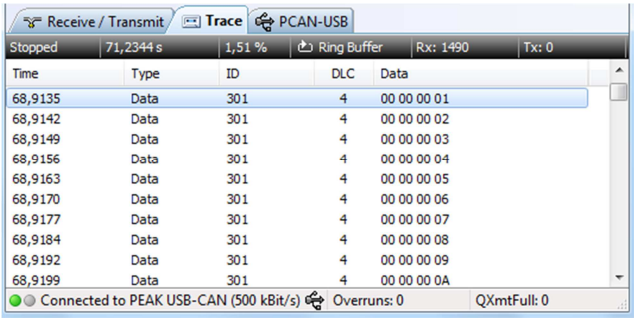


Figure : Exemple de log de l'application PCAN-View

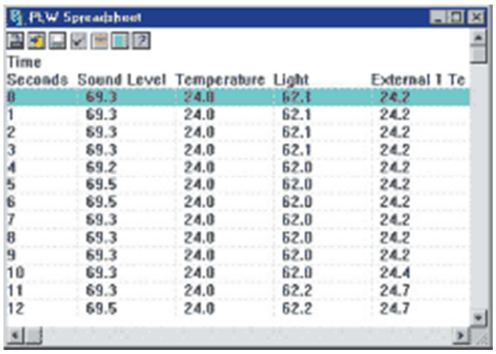


Figure : Exemple de log de l'application PicoLog

PCAN-View

Temps	Status
0,015	0x300
0,030	0x301
0,045	0x302
0,060	0x303

Table : Status

Oscilloscope

Temps	Alimentation	Courant
0,015	+10V	
0,030	+10V	
0,045	+10V	
0,060	+10V	

Table : Voltage et courant

PicoLog

Temps	T°C	U1 T°C	U2 T°C	U3 T°C
0,01	25°C			
0,02	25°C			
0,03	25°C			
0,04	25°C			

Table : Temperatures mesures dans le temps

Moyens d'essais et configuration hardware et software (CONFIGURATION 2)

L'idée et récupérer toute l'information dans le logiciel PicoLog qui proportionne une graphique unique avec les valeurs des paramètres mesures. Pour .la réalisation de ce test on aura besoin des suivants dispositifs:

- Une **source d'alimentation stabilisée** a voltage variable entre +8 et +12v.
- Un **analyseur de bus CAN** (pour recevoir les trames indican le fonctionnement et statuts de l'EDR)
- Una **unité de mesure USB TC-08** Thermocouple Data Logger du fabricant Pico Technology

  http://www.picotech.com/thermocouple.html

  The TC-08 Thermocouple Data Logger offers industry–leading performance and is an inexpensive answer to your temperature measuring needs.

  - *8 channel thermocouple data logger*
  - *Measure from 1 to 160 thermocouples*
  - *Measures from -270 to +1820 °C*
  - *Resolution 20 bits (16.25 NFR)*
  - *Conversion time - per active channel 100 ms*
- Des cartes terminal pour mesurer alimentation et courant **Single Channel Terminal Board** du fabricant Pico Technology. C'est un accessoire de l'unité de mesure USB TC-08.

  The USB TC-08 Single-Channel Terminal Board is an accessory for the USB TC-08 Thermocouple Data Logger. The screw terminals allow wires to be attached to the data logger without soldering and enable the USB TC-08 to measure voltages from 0 to +5 V, or 4-20 mA loop current.

- Des **sondes de température** de type XF-321-FAR - CAPTEUR T/COUPLE **K** PTFE 1M MINIPLUG du fabricant LABFACILITY et distribue par Farnell.

  http://fr.farnell.com/labfacility/xf-321-far/capteur-t-couple-k-ptfe-1m-miniplug/dp/4100748

  Type K Thermocouple (Exposed wire, PTFE insulated)
  Tip Diameter: 1.5 mm
  Tip Temperature: -75 to +250 °C

  
- Un PC de control pour récupérer de l'information pendant le test avec les logiciels :
  - **PCAN-View** qui permettra d'obtenir un log avec le temps de réception de chaque trame. Celci permet de démontrer que l'application tourne dans l'EDR et que les composants fonctionnent dans de bonnes conditions.
  - **PicoLog** qui registrera également dans un log les températures des composants mesures a un intervalle de temps que l'on pourra fixer.

    http://www.picotech.com/data-logging-software.html

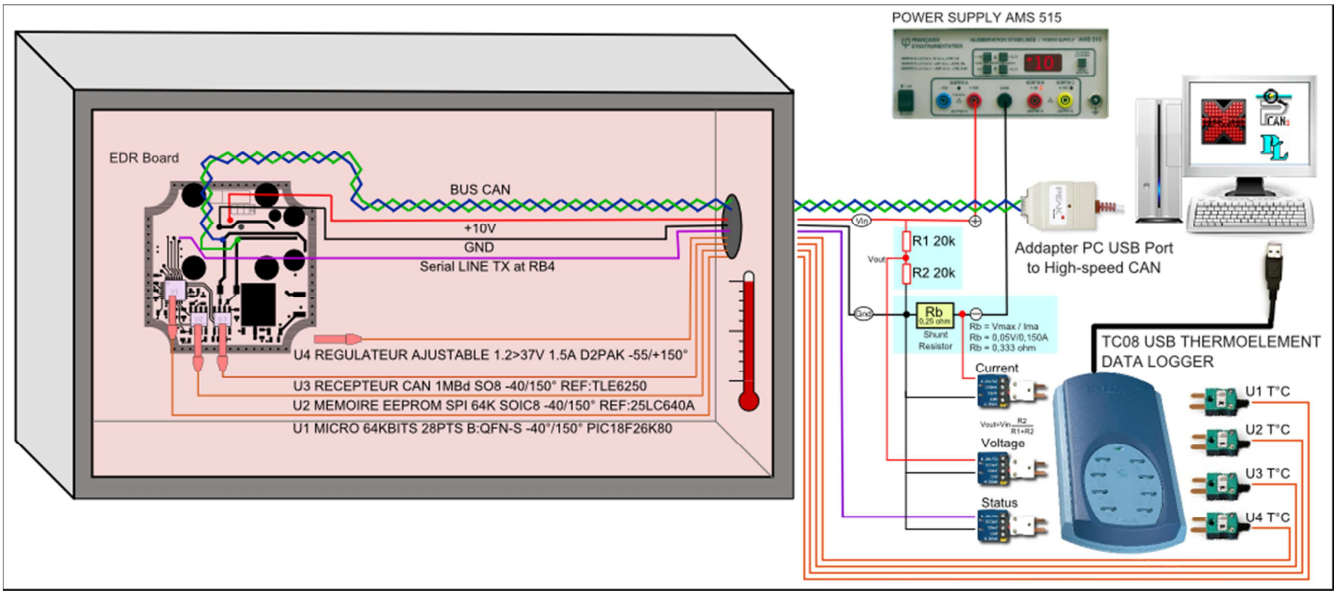


Figure : Connexions entre la carte EDR et les différents éléments de log et mesure.

Mesure de voltages et courant

L'USB TC-08 Single-Channel Terminal Board est un accessoire pour le dispositif Pico Technology USB TC-08 8-Channel Thermocouple Data Logger. Les terminaux à vis vont permettre la connexion des câbles sans soudure et mesurer voltages entre 0 et +05V ou bien boucles de courant de 4 à 20 mA.

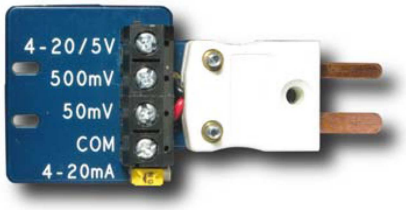


Figure: USB TC-08 Single-Channel Terminal Board

Name	Function	Voltage displayed in PicoLog
4-20/5V	Input for the positive side of a 4-20 mA loop or 0 to 5 V signal	0 – 50 mV
500mV	Input for the positive side of a 0 to 500 mV signal	0 – 50 mV
50mV	Input for the positive side of a 0 to 50 mV signal	0 – 50 mV
COM	Input for the negative side of any voltage or current signal.	-
4-20 mA	Set switch to "ON" for a 4-20 mA loop signal. Set switch to "OFF" for all voltage signals.	9.6 – 48 mV

Measuring voltages and currents

1. Connect the negative side of your circuit to the COM terminal.
2. Connect the positive side of your circuit to the "5 V", "500 mV" or "50 mV" terminal depending on the voltage range of the signal. For a 4-20 mA loop signal, use the "5 V" input.
3. For a 4-20 mA loop signal, set the switch to "ON". For a voltage signal, set the switch to "OFF".
4. Plug the terminal board into the USB TC-08.
5. Connect the USB TC-08 to the computer using the USB cable supplied with the logger.
6. Run PicoLog on the computer.
7. Go to File -> New Settings.
8. Set the converter type to USB TC-08.
9. Edit one of the USB TC-08 channels and set the "Thermocouple" control to "mV".
10. PicoLog will display the voltage or current applied to the terminal board in the monitor window.

Moyens d'essais et configuration hardware et software (CONFIGURATION 3)

Etant donné que le dispositif Pico Data Logger nécessite un **temps de conversion de 100ms pour chaque sonde** et que il tient un seule convertisseur ADC, pour obtenir 4 mesures de température pour 4 composants il nécessitera de 400ms, cela veut dire que pour un seul composant on aura mesures valables que tous les 400ms.

Pour les mesures de voltage ce pas grave puisque en principe on ne le fera pas varier lors d'un essai, mais pour les mesures de courant cet intervalle est assez grand et on perd beaucoup de résolution. Donc le principe de cette configuration consiste à:

- **Réduire le nombre de sondes** pour gagner en fréquence des mesures de température. Dans l'schéma suivant on enlève la sonde de température de l'EEPROM et on maintien la entrée de statuts afin de pouvoir repérer les mesures de température dans le temps, sachant que il peut avoir un décalage de 400 msec (4 sondes * 100 ms/sonde)
- **Utiliser l'oscilloscope** pour gagner en précision dans les **mesures de courant**. Pour cela on utilise une résistance afin de traduire le courant dans une tension mesurable, sur la quel il faudra tenir en compte.
    - **L'emplacement de la résistance**. Sur la ligne de retour de **masse** de l'EDR vers la source d'alimentation (La placer sur la ligne +10V pourra créer un court-circuit avec la masse de l'oscilloscope).
    - **La valeur**. Selon calcul pour mesurer su une échelle de 50mV avec une consommation maxime de 150mA on devrait utiliser R= Vmax / Imax = 0,05V / 0,150 A = 0,33 ohm. En prenant une marge de sécurité on utilisera donc une résistance de **0,25ohm**.
    - **Les points de mesure de la sonde** de l'oscilloscope. On placera la **pointe de mase** dans un point situe entre la carte EDR et la résistance, et la **pointe de mesure** entre la résistance et la source d'alimentation, ce que nous donnera des valeurs négatives à traduire.

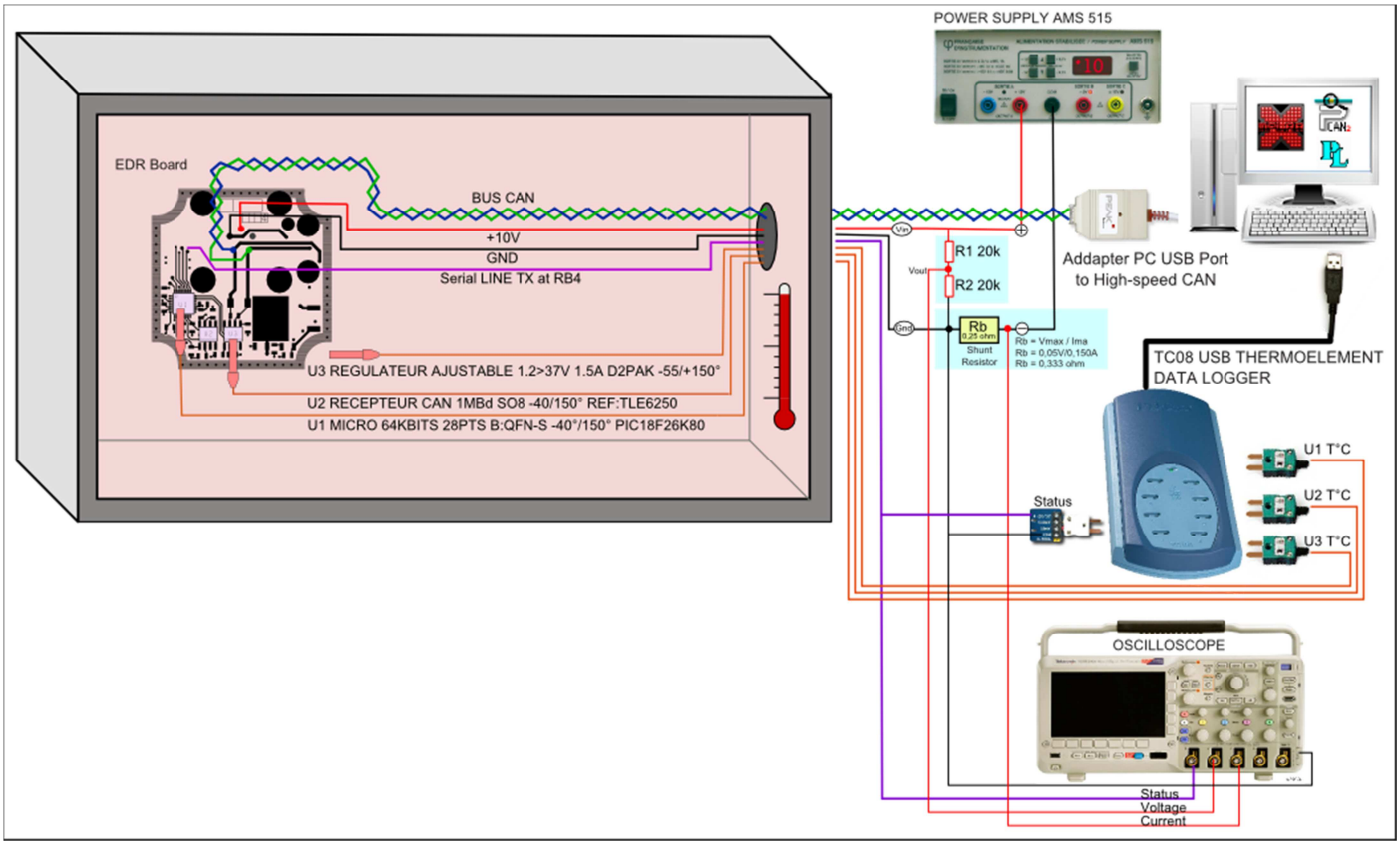


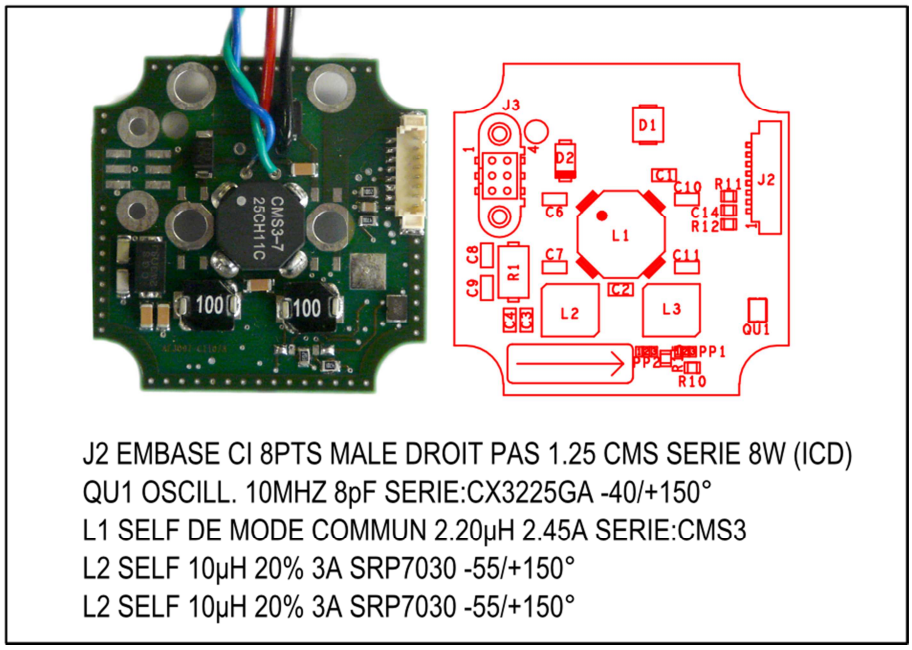
Figure : Connexions entre la carte EDR et les différents éléments de log et mesure.

4.2 Test étanchéité

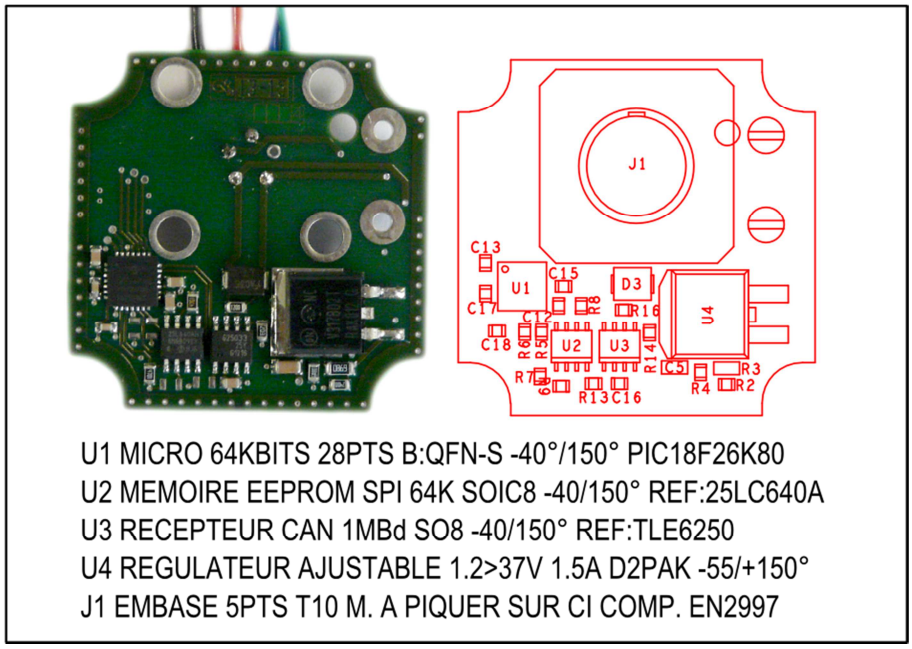
4.3 Test vibrations

5 Appendice

5.1 Appendice 1. Carte EDR



J2 EMBASE CI 8PTS MALE DROIT PAS 1.25 CMS SERIE 8W (ICD)
QU1 OSCILL. 10MHZ 8pF SERIE:CX3225GA -40/+150°
L1 SELF DE MODE COMMUN 2.20µH 2.45A SERIE:CMS3
L2 SELF 10µH 20% 3A SRP7030 -55/+150°
L2 SELF 10µH 20% 3A SRP7030 -55/+150°



U1 MICRO 64KBITS 28PTS B:QFN-S -40°/150° PIC18F26K80
U2 MEMOIRE EEPROM SPI 64K SOIC8 -40/150° REF:25LC640A
U3 RECEPTEUR CAN 1MBd SO8 -40/150° REF:TLE6250
U4 REGULATEUR AJUSTABLE 1.2>37V 1.5A D2PAK -55/+150°
J1 EMBASE 5PTS T10 M. A PIQUER SUR CI COMP. EN2997

5.2 Appendice 2. Problème de clock avec compilateur xC8

Freq	Frec/4	/1000000	1/x
	Int/sec	Inst/us	us/inst
1.000.000	250.000	0.25	4
2.000.000	500.000	0.5	2
4.000.000	1.000.000	1	1
8.000.000	2.000.000	2	0.5
10.000.000	2.500.000	2.5	0.4
16.000.000	4.000.000	4	0.25
20.000.000	5.000.000	5	0.2
25.000.000	6.250.000	6.25	0.16
30.000.000	7.500.000	7.5	0.1333
32.000.000	8.000.000	8	0.125

	PICDEM CAN-LIN 3 (25MHz)							
Funtion			One (Osc .M)		Zero	Delta	Period	Delta
_delay(1)	0001		640ns		0,32 us		0,960 us	
_delay(2)	0010		800ns (799,1ns)	0,160 us	0,48 us	0,160 us	1,280 us	0,32 us
_delay(3)	0011		960ns (959.0ns)	0,160 us	0,64 us	0,160 us	1,600 us	0,32 us
_delay(4)	0100		1,120 (1,119 us)	0,160 us	0,80 us	0,160 us	1,920 us	0,32 us
_delay(5)	0101		1,128 (1,279 us)	0,160 us	0,96 us	0,160 us	2,240 us	0,32 us
_delay(6)	0110		1,440 (1,439 us)	0,160 us	1,12 us	0,160 us	2,560 us	0,32 us
_delay(7)	0111		1,440 (1,439 us)	0 us	1,12 us	0 us	2,560 us	0 us
_delay(8)	1000		1,440 (1,439 us)	0 us	1,12 us	0 us	2,560 us	0 us
_delay(9)	1001		1,600 (1,599 us)	0,160 us	1,28 us	0,160 us	2,880 us	0,32 us
_delay(10)	1010		1,760 (1,759us)	0,160 us	1,44 us	0,160 us	3,200 us	0,32 us
_delay(11)	1011		1,920 (1,919 us)	0,160 us	1,60 us	0,160 us	3,520 us	0,32 us
_delay(12)	1100		1,920 (1,919 us)	0 us	1,60 us	0 us	3,520 us	0 us
_delay(13)	1101		2,080 (2,079 us)	0,160 us	1,78 us	0,620 us	3,840 us	0,32 us
_delay(14)	1110		2,240 (2,239 us)	0,160 us	1,90 us	0,120 us	4,160 us	0,32 us
_delay(15)	1111		2,400 (2,399 us)	0,160 us	2,06 us	0,160 us	4,480 us	0,32 us
_delay(16)	1 0000		2,400 (2,399 us)	0 us	2,06 us	0 s	4,480 us	0 us
_delay(17)	1 0001		2,560 (2,599 us)	0,160 us	2,22 ns	0,160 us	4,800 us	0,32 us
_delay(18)	1 0010		2,720 (2,719 us)	0,160 us	2,38 ns	0,160 us	5,120 us	0,32 us
_delay(19)	1 0011		2,880 (2,879 us)	0,160 us	2,54 ns	0,160 s	5,440 us	0,32 us
_delay(20)	1 0100		(2,879 us)	0 us	2,54	0 us	5,440 us	0 us
_delay(25)			(3,679 us)		3,34 ns		7,040 us	
_delay(31)			(4,479 us)		4,18 us		8,640 us	
_delay(37)								
	PICDEM CAN-LIN 3 (25MHz) __delay_us(x) _delay((unsigned long)((x)*(_XTAL_FREQ/4000000.0)))							
__delay_us(1)	1*6,25 = 06,25		(1,439 us)		1,12 us		2,560us	
__delay_us(2)	2*6,25 = 12,50		(1,919 us)	0,48	1,60 us	0,48	3,520 us	0,96
__delay_us(3)	3*6,25 = 18,75		(2,719 us)	0,8	2,40 us	0,8	5,120 us	1,6
__delay_us(4)	4*6,25 = 25,00		(3,679 us)	0,96	3,36 us	0,96	7,040 us	1,92
__delay_us(5)	5*6,25 = 31,25		(4,479 us)	0,799	4,18 us	0,82	8,640 us	1,6
__delay_us(6)	6*6,25 = 37,50		(4,959 us)	0,48	4,64 us	0,46	9,600 us	0,96
__delay_us(10)	10*6,25 = 62,5		(7,999 us)		7,68		15,68 us	
__delay_us(100)	100*6,25 = 625		(80,48 us)		80,0 us		160,6	
__delay_us(1000)	1000*6,25 = 6250		(751,2 us)		748 us		1.502 ms	
__delay_us(2000)	2000*6,25 = 12500							
__delay_us(3000)								
__delay_us(4000)								
__delay_us(5000)	5000*6,25 = 31215		(3,754 ms)		3,75 ms		7,506 ms	
	PICDEM CAN-LIN 3 (25MHz) __delay_ms(x) _delay((unsigned long)((x)*(_XTAL_FREQ/4000.0)))							
__delay_ms(1)	1*6250 = 6250		(751,2 us)		748 us		1,502 ms	
__delay_ms(2)	2*6250 = 12500							
__delay_ms(3)	3*6250 = 18750							
__delay_ms(4)	4*6250 = 25000							
__delay_ms(5)	5*6250 = 31250		(3,754 ms)		3,75 ms		7,506 ms	
__delay_ms(10)	10*6250 = 62500		(7,508 ms)		7,52 ms		15,01 ms	
__delay_ms(20)	20*6250 = 125000		(15,02 ms)		15,0 ms		30,02 ms	
__delay_ms(100)	Too Large							
__delay_ms(1000)	Too Large							

```
147:                    _delay(1);
DF34  F000     NOP                (1)
```

```
147:                    _delay(2);
DF34  D000     BRA 0xDF36   (2)
```

```
147:                    _delay(3);
DF30  F000     NOP          (1)
DF32  D000     BRA 0xDF34   (2)
```

```
147:                    _delay(4);
DF30  D000      BRA 0xDF32          (2)
DF32  D000      BRA 0xDF34          (2)
```

```
147:                    _delay(5);
DF94  F000      NOP             (1)
DF96  D000      BRA 0xDF98      (2)
DF98  D000      BRA 0xDF9A      (2)
```

```
147:                    _delay(6);
DF94  D000      BRA 0xDF96      (2)
DF96  D000      BRA 0xDF98      (2)
DF98  D000      BRA 0xDF9A      (2)
```

```
147:                    _delay(7);
DF8C  0E01      MOVLW 0x1
DF8E  F000      NOP
DF90  2EE8    DECFSZ WREG, F, ACCESS
DF92  D7FD    BRA 0xDF8E
DF94  D000    BRA 0xDF96        T 6 !
```

```
147:                    _delay(8);
DF94  0E02 MOVLW 0x2
DF96  2EE8 DECFSZ WREG, F, ACCESS
DF98  D7FE BRA 0xDF96    T 6 !!!!
```

```
147:                    _delay(9);
DF90  0E02    MOVLW 0x2
DF92  2EE8    DECFSZ WREG, F, ACCESS
DF94  D7FE    BRA 0xDF92
DF96  F000    NOP             T 7 !!!!
```

```
147:                    _delay(10);
DF90 0E02   MOVLW 0x2
DF92 F000   NOP
DF94 2EE8   DECFSZ WREG, F, ACCESS
DF96 D7FD   BRA 0xDF92   T 8 !
```

```
147:                    _delay(11);
DF8C  0E02      MOVLW 0x2
DF8E  F000      NOP
DF90  2EE8      DECFSZ WREG, F, ACCESS
DF92  D7FD      BRA 0xDF8E
DF94  F000      NOP    T 9 !
```

```
147:                    _delay(12);
DF94  0E03      MOVLW 0x3
DF96  2EE8      DECFSZ WREG, F, ACCESS
DF98  D7FE      BRA 0xDF96     T 9 !
```

_delay(1)	0001		640ns		0,32 us		0,960 us	
_delay(2)	0010		800ns (799,1ns)	0,160 ns	0,48 us	0,160 ns	1,280 us	0,32 us
_delay(3)	0011		960ns (959.0ns)	0,160 ns	0,64 us	0,160 ns	1,600 us	0,32 us
_delay(4)	0100		1,120 (1,119 us)	0,160 ns	0,80 us	0,160 ns	1,920 us	0,32 us
_delay(5)	0101		1,128 (1,279 us)	0,160 ns	0,96 us	0,160 ns	2,240 us	0,32 us
_delay(6)	0110		1,440 (1,439 us)	0,160 ns	1,12 us	0,160 ns	2,560 us	0,32 us
_delay(7)	0111		1,440 (1,439 us)	0 ns	1,12 us	0 ns	2,560 us	0 ns
_delay(8)	1000		1,440 (1,439 us)	0 ns	1,12 us	0 ns	2,560 us	0 ns
_delay(9)	1001		1,600 (1,599 us)	0,160 ns	1,28 us	0,160 ns	2,880 us	0,32 ns
_delay(10)	1010		1,760 (1,759us)	0,160 ns	1,44 us	0,160 ns	3,200 us	0,32 ns
_delay(11)	1011		1,920 (1,919 us)	0,160 ns	1,60 us	0,160 ns	3,520 us	0,32 ns
_delay(12)	1100		1,920 (1,919 us)	0 us	1,60 us	0 us	3,520 us	0 us

```
147:                    __delay_us(1);
DF94  D000      BRA 0xDF96
DF96  D000      BRA 0xDF98
DF98  D000      BRA 0xDF9A
```

```
147:                    __delay_us(2);
DF94  0E03      MOVLW 0x3
DF96  2EE8      DECFSZ WREG, F, ACCESS
DF98  D7FE      BRA 0xDF96
```

```
147:                    __delay_us(3);
E290  0E04      MOVLW 0x4
E292  2EE8      DECFSZ WREG, F, ACCESS
E294  D7FE      BRA 0xE292
E296  D000      BRA 0xE298
```

```
147:                    __delay_us(4);
DF90  0E05      MOVLW 0x5
DF92  F000      NOP
DF94  2EE8      DECFSZ WREG, F, ACCESS
DF96  D7FD      BRA 0xDF92
```

```
147:                    __delay_us(5);
DF8C  0E06      MOVLW 0x6
DF8E  F000      NOP
DF90  2EE8      DECFSZ WREG, F, ACCESS
DF92  D7FD      BRA 0xDF8E
DF94  F000      NOP
```

```
147:                    __delay_us(6);
DF90  0E09      MOVLW 0x9
DF92  2EE8      DECFSZ WREG, F, ACCESS
DF94  D7FE      BRA 0xDF92
DF96  F000      NOP
```

```
147:                    __delay_us(7);
DF8C  0E0A      MOVLW 0xA
DF8E  2EE8      DECFSZ WREG, F, ACCESS
DF90  D7FE      BRA 0xDF8E
DF92  D000      BRA 0xDF94
DF94  F000      NOP
```

```
147:                    __delay_us(8);
DF90  0E0A      MOVLW 0xA
DF92  F000      NOP
DF94  2EE8      DECFSZ WREG, F, ACCESS
DF96  D7FD      BRA 0xDF92
```

```
147:                    __delay_us(9);
DF94  0E0E      MOVLW 0xE
DF96  2EE8      DECFSZ WREG, F, ACCESS
DF98  D7FE      BRA 0xDF96
```

```
147:                    __delay_us(10);
DF90  0E0F      MOVLW 0xF
DF92  2EE8      DECFSZ WREG, F, ACCESS
DF94  D7FE      BRA 0xDF92
DF96  D000      BRA 0xDF98
```

```
147:                    __delay_us(100);
DF90  0E7D      MOVLW 0x7D
DF92  F000      NOP
DF94  2EE8      DECFSZ WREG, F, ACCESS
DF96  D7FD      BRA 0xDF92
```

```
147:                    __delay_us(1000);
DF80  0E07      MOVLW 0x7
DF82  6E2B      MOVWF 0x2B, ACCESS
DF84  0E16      MOVLW 0x16
DF86  2EE8      DECFSZ WREG, F, ACCESS
DF88  D7FE      BRA 0xDF86
DF8A  2E2B      DECFSZ 0x2B, F, ACCESS
DF8C  D7FC      BRA 0xDF86
DF8E  F000      NOP
```

__delay_us(1)	1*6,25 = 06,25		(1,439 us)		1,12 us		2,560us	
__delay_us(2)	2*6,25 = 12,50		(1,919 us)	0,48	1,60 us	0,48	3,520 us	0,96
__delay_us(3)	3*6,25 = 18,75		(2,719 us)	0,8	2,40 us	0,8	5,120 us	1,6
__delay_us(4)	4*6,25 = 25,00		(3,679 us)	0,96	3,36 us	0,96	7,040 us	1,92
__delay_us(5)	5*6,25 = 31,25		(4,479 us)	0,799	4,18 us	0,82	8,640 us	1,6
__delay_us(6)	6*6,25 = 37,50		(4,959 us)	0,48	4,64 us	0,46	9,600 us	0,96
__delay_us(10)	10*6,25 = 62,5		(7,999 us)		7,68		15,68 us	

```
147:                    __delay_ms(1);
DF80  0E07      MOVLW 0x7
DF82  6E2B      MOVWF 0x2B, ACCESS
DF84  0E16      MOVLW 0x16
DF86  2EE8      DECFSZ WREG, F, ACCESS
DF88  D7FE      BRA 0xDF86
DF8A  2E2B      DECFSZ 0x2B, F, ACCESS
DF8C  D7FC      BRA 0xDF86
DF8E  F000      NOP
```

```
147:                    __delay_ms(10);
DF7C  0E3D      MOVLW 0x3D
DF7E  6E2B      MOVWF 0x2B, ACCESS
DF80  0EE9      MOVLW 0xE9
DF82  2EE8      DECFSZ WREG, F, ACCESS
DF84  D7FE      BRA 0xDF82
DF86  2E2B      DECFSZ 0x2B, F, ACCESS
DF88  D7FC      BRA 0xDF82
DF8A  D000      BRA 0xDF8C
DF8C  F000      NOP
```

```
147:                    __delay_ms(20);
DF80  0E7A      MOVLW 0x7A
DF82  6E2B      MOVWF 0x2B, ACCESS
DF84  0ED4      MOVLW 0xD4
DF86  2EE8      DECFSZ WREG, F, ACCESS
DF88  D7FE      BRA 0xDF86
DF8A  2E2B      DECFSZ 0x2B, F, ACCESS
DF8C  D7FC      BRA 0xDF86
DF8E  F000      NOP
```